Recommendation Systems

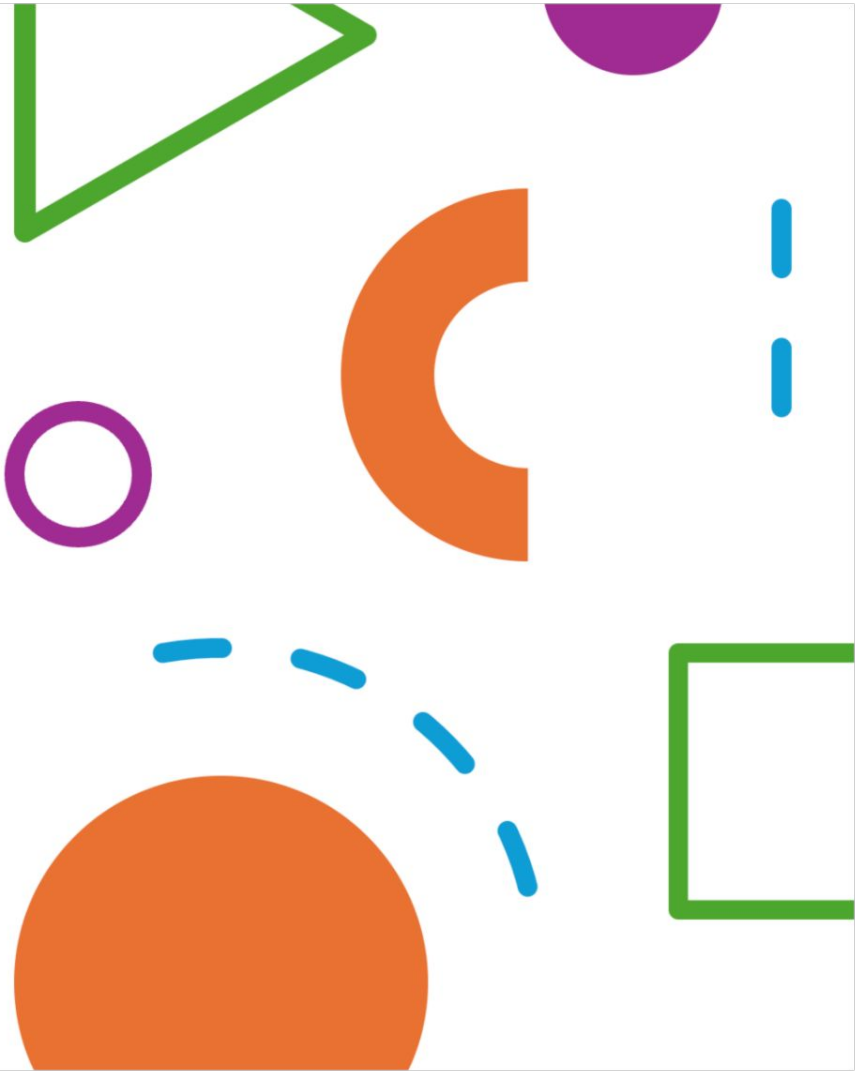
Part 2

Lê Thiện Hòa

Contents

- Implicit Feedback, Data Sparsity & Evaluation
- ALS (**Retrieval**) & BPR (**Ranking**)
- Word2vec & NeuMF
- Two-Tower, Wide-and-Deep, DeepFM
- **Graph-based RecSys**: Metapath2vec & GraphSAGE
- **Multi-Armed Bandit** (MAB), Contextual Bandit, Clustered Bandit
- **Sequential Model**: Bert4rec
- **LLMs** for RecSys

Session 6:
Multi-Armed Bandits, Contextual Bandits, Clustered Bandits



Agenda overview:

- Explore–exploit fundamentals
- Multi-Armed Bandit
  - eps-greedy
  - UCB
  - Thompson Sampling
- Contextual Bandit
  - Linear UCB
  - LinTS
- Clustered Bandit
  - User segmentation
  - Sharing exploration among clusters

Motivation

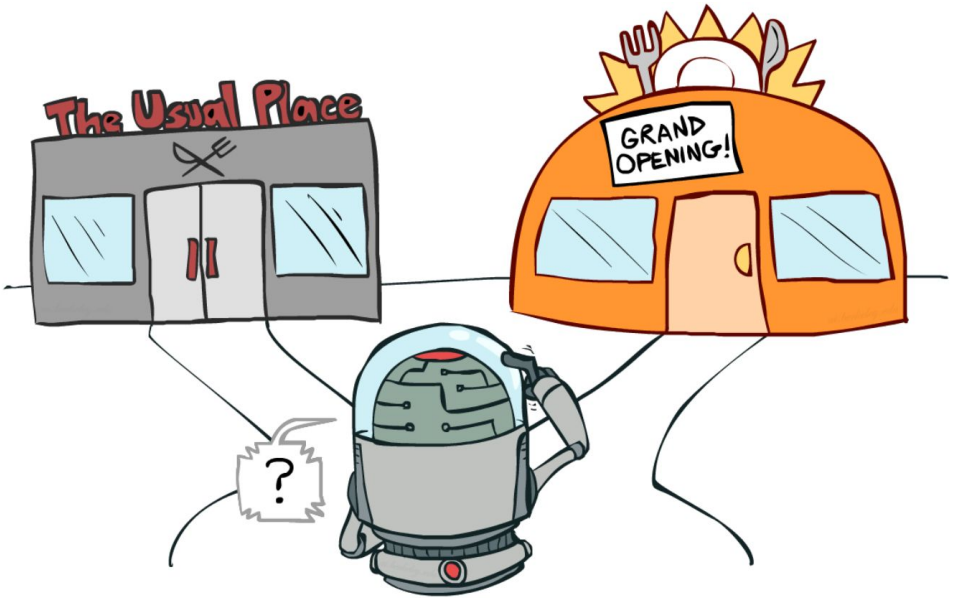
- Real-world RecSys must balance:
  - Showing **what we know works** (exploit)
  - Trying **new or uncertain options** (explore)
- Examples:
  - Netflix recommending a new movie
  - E-commerce promoting a new product
  - News feed ranking fresh articles

Explore vs Exploit

- **Exploit:** choose the best-known action
- **Explore:** try uncertain actions to gain info
- Tradeoff: short-term reward vs long-term learning

Example: Restaurant choice

- Go to favorite place (exploit)
- Try new place (explore)

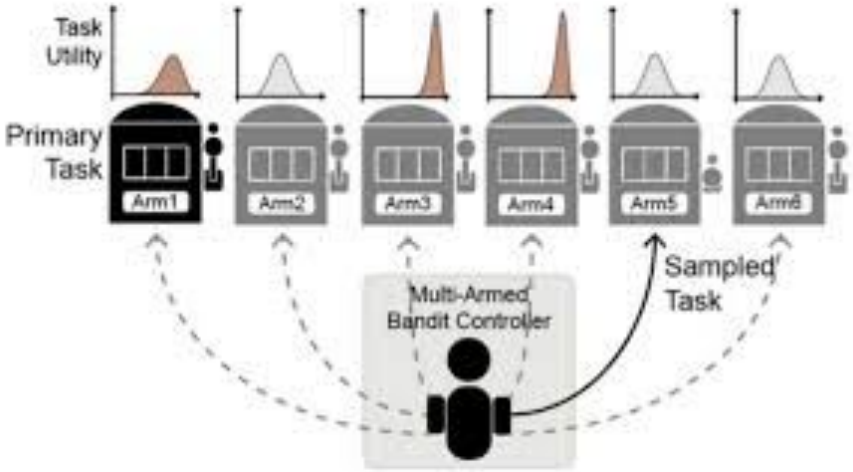


Multi-Armed Bandit (MAB)

Formal Problem Setup

- We have K actions (arms)
- Each action gives stochastic reward
- Goal: maximize cumulative reward over time

Key challenge: unknown reward distributions



Multi-Armed Bandit (MAB)

- Named after slot machines ("one-armed bandits")
- At each step:
  1. Choose an arm
  2. Observe reward
  3. Update belief

Assumption: reward depends only on the arm (no context)

Notation

- First, we denote the reward (that is, 1 for a click, 0 for no click) received after selecting the action, a, for the ith time by R_i.
- The average reward observed before the nth selection of this same action is defined as follows:

$$Q_n \triangleq \frac{R_1 + R_2 + \cdots + R_{n-1}}{n-1}$$

  This estimates the expected value of the reward that this action yields, R, after $n-1$ observations.
- This is also called the **action value** of a. Here, this is Q_n estimates of the action value after selecting this action $n-1$ times.
- Now, we need a bit of simple algebra and we will have a very convenient formula to update the action values:

$$\begin{aligned} Q_{n+1} &= \frac{R_1 + R_2 + \cdots + R_n}{n} \\ &= \frac{R_1 + R_2 + \cdots + R_{n-1}}{n} + \frac{R_n}{n} \\ &= \frac{n-1}{n-1} \cdot \frac{R_1 + R_2 + \cdots + R_{n-1}}{n} + \frac{R_n}{n} \end{aligned}$$

Notation

$$= \frac{n-1}{n} \cdot \frac{R_1 + R_2 + \cdots + R_{n-1}}{n-1} + \frac{R_n}{n}$$

$$= \frac{n-1}{n} \cdot Q_n + \frac{R_n}{n}$$

$$= Q_n + \frac{1}{n} \cdot (R_n - Q_n)$$

- Remember that Q_n is our estimate for the action value of a before we take it for the nth time. When we observe the reward, R_n, it gives us another signal for the action value. We don't want to discard our previous observations, but we also want to update our estimate to reflect the new signal.

  So, we adjust our current estimate, Q_n, in the direction of the **error** that we calculate based on the latest observed reward, $R_n - Q_n$, with a **step size** of $1/n$ and obtain a new estimate, Q_{n+1}. This means, for example, if the latest observed reward is greater than our current estimate, we revise the action value estimate upward.

- For convenience, we define $Q_0 \triangleq 0$.

Notation

- Notice that the rate at which we adjust our estimate will get smaller as we make more observations due to the $1/n$ term. So, we put less weight on the most recent observations and our estimate for the action value for a particular action will settle down over time.
- However, this might be a disadvantage if the environment is not stationary but is changing over time. In those cases, we would want to use a step size that does not diminish over time, such as a fixed step size of $\alpha \in (0,1)$.
- Note that this step size must be smaller than 1 for the estimate to converge (and larger than 0 for a proper update).
- Using a fixed value for α will make the weights of the older observations decrease exponentially as we take action α more and more.

Let's bring α back to the notation, so we can obtain our formula to update the action values:

$$Q_{n+1}(a) = Q_n(a) + \alpha(R_n(\alpha) - Q_n(a))$$

Here, α is a number between 0 and 1. For stationary problems, we usually set $\alpha = 1/N(a)$, where $N(a)$ is the number of times the action a has been taken up to that point (which was denoted by n initially). In stationary problems, this will help action values converge quicker, due to the diminishing $1/N(a)$ term, rather than chasing after noisy observations.

ε-Greedy Strategy

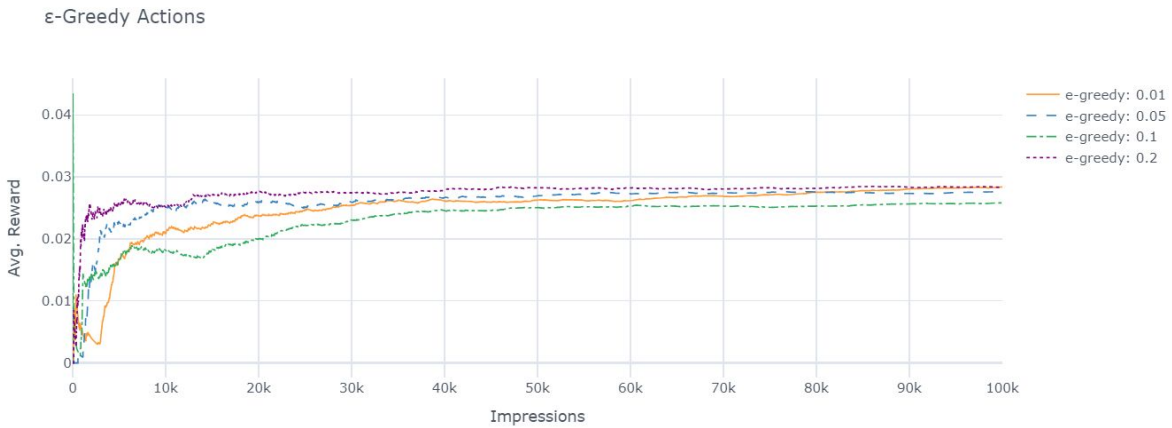
- With probability ε: explore (random arm)
- With probability 1−ε: exploit (best estimated arm)

Pros:

- Simple
- Easy to implement

Cons:

- Exploration is random
- No notion of uncertainty



Upper Confidence Bound (UCB1)

Principle: optimism under uncertainty

Formula:

$$A_t = \text{argmax}_a \left(Q_t(a) + c \sqrt{\frac{\ln(t)}{N_t(a)}} \right)$$

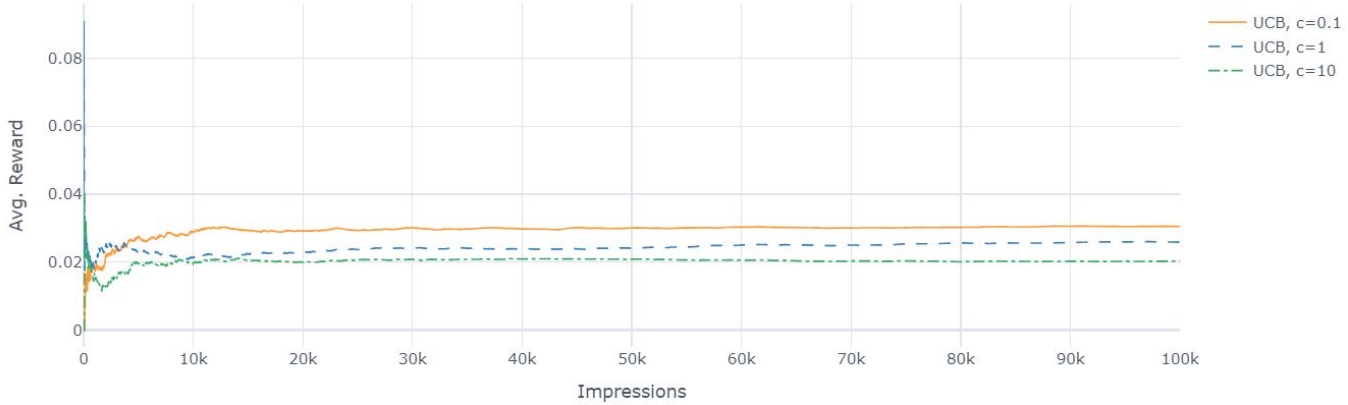
Exploit

Explore

t = timesteps

$N_t(a)$ = no. of times action (a) is taken

Action Selection using UCB. Best avg. reward: 0.0305



Thompson Sampling (TS)

Bayesian approach:

- Maintain posterior distribution for each arm
- Sample from each posterior
- Choose arm with highest sampled reward

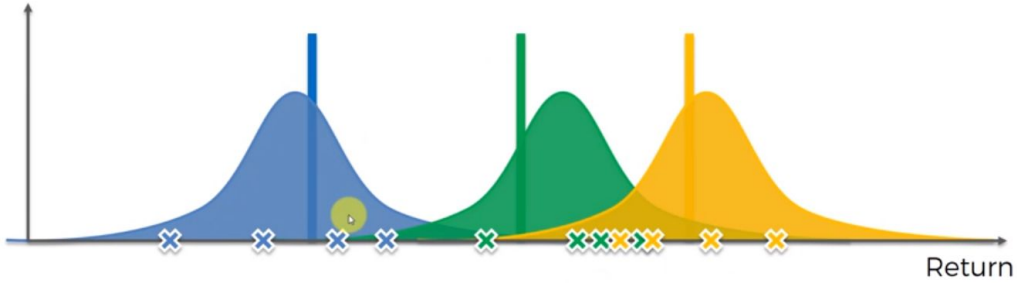
TS Intuition

- Probability matching
- Naturally balances explore & exploit

Benefits:

- Works very well in practice
- Simple once Bayesian model is defined

$$p(\theta|X) = \frac{p(X|\theta)p(\theta)}{p(X)}$$



The beta distribution for the prior of the ad k is given by the following:

$$p(\theta_k) = \frac{\Gamma(\alpha_k + \beta_k)}{\Gamma(\alpha_k)\Gamma(\beta_k)} \theta_k^{\alpha_k - 1}(1 - \theta_k)^{\beta_k - 1}$$

Here, α_k and β_k are the parameters characterizing the beta distribution, and $\Gamma(\cdot)$ is the gamma function.
To initialize the prior, we use $\alpha_k = \beta_k = 1$, which makes θ_k uniformly distributed over $[0, 1]$. Once we observe a reward, R_t, after selecting the ad k, we obtain the posterior distribution, as follows:

$$\alpha_k \leftarrow \alpha_k + R_t \; s \; \beta_k \leftarrow \beta_k + 1 - R_t$$

Thompson Sampling (TS)

Instead of estimating each arm with a single fixed number (as in UCB),

Thompson Sampling treats the true mean reward of each arm as a random variable.

Assume:

$$r \sim \text{Bernoulli}(\theta_a)$$

where θ_a is the unknown success probability of arm a.

We place a prior:

$$\theta_a \sim \text{Beta}(\alpha_a, \beta_a)$$

Reasons for using Beta:

- Support is $[0, 1]$, matching probability values
- It is the **conjugate prior** of the Bernoulli distribution
- The posterior distribution remains Beta after observing data
- Leads to simple and exact Bayesian updates (no approximation needed)

Thompson Sampling (TS)

Given:

- Prior: $\theta_a \sim \text{Beta}(\alpha, \beta)$
- Observations:
  - S successes (reward = 1)
  - F failures (reward = 0)

Likelihood:

$$p(\text{data} \mid \theta_a) = \theta_a^S (1 - \theta_a)^F$$

Prior:

$$p(\theta_a) = \theta_a^{\alpha-1} (1 - \theta_a)^{\beta-1}$$

Posterior (Bayes rule):

$$p(\theta_a \mid \text{data}) \propto \theta_a^{\alpha+S-1} (1 - \theta_a)^{\beta+F-1}$$

Therefore:

$$\theta_a \mid \text{data} \sim \text{Beta}(\alpha + S, \beta + F)$$

Thompson Sampling (TS)

Practical Update Rule

Typically initialize:

$$\alpha_a = 1, \quad \beta_a = 1 \quad \text{(uniform prior)}$$

After pulling arm a:

- If reward = 1:

$$\alpha_a \leftarrow \alpha_a + 1$$

- If reward = 0:

$$\beta_a \leftarrow \beta_a + 1$$

Action Selection Rule

At timestep t:

For each arm a:

$$\tilde{\theta}_a \sim \text{Beta}(\alpha_a, \beta_a)$$

Choose:

$$a_t = \arg\max_a \tilde{\theta}_a$$



ε-Greedy Strategy *vs.* UCB1 *vs.* Thompson Sampling

Method	Exploration	Complexity	Performance
ε-greedy	Random	Very low	Medium
UCB1	Deterministic, uncertainty-based	Low	Good
Thompson	Bayesian sampling	Medium	Very good

Contextual Bandit

- Reward depends on:
  - Action
  - Context (user, time, device, etc.)

At each step:

- Observe context x
- Choose action a
- Get reward r

Contextual Bandit

While solving (contextual) multi-armed bandit problems, our goal is to learn action values for each arm (action) from our observations, which we have denoted by $Q(a)$. In the online advertising example, it represented our estimate for the probability of a user clicking the ad if we displayed a. Now, assume that we have two pieces of information about the user seeing the ad, namely the following:

- Device type (mobile or desktop)
- Location (domestic/US or international/non-US)

It is quite likely that ad performances will differ by device type and location, which make up the context in this example. A CB model will therefore leverage this information, estimate the action values for each context, and choose the actions accordingly.

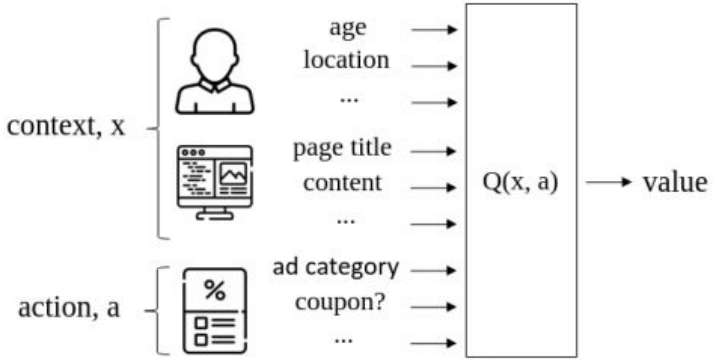
This would look like filling in a table for each ad similar to the following:

$Q(a = D)$	Domestic	International
Mobile	0.031	0.02
Desktop	0.036	0.022

Table 3.1 – Sample action values for ad D

This means solving four MAB problems, one for each context:

- Mobile – Domestic
- Mobile – International
- Desktop – Domestic
- Desktop – International



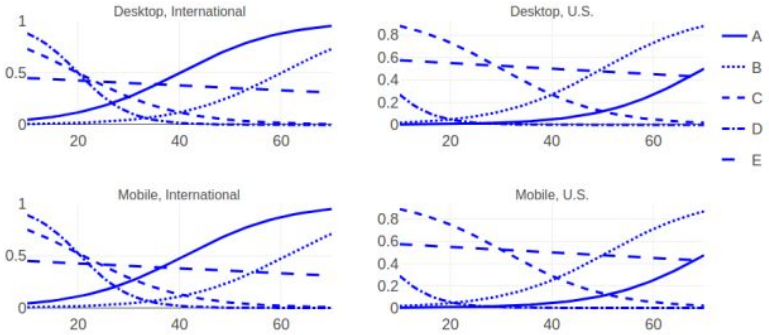


Figure 3.2 – Comparison of the true ad click probabilities given the context (x axis: age, y axis: click probability)

Contextual Bandit

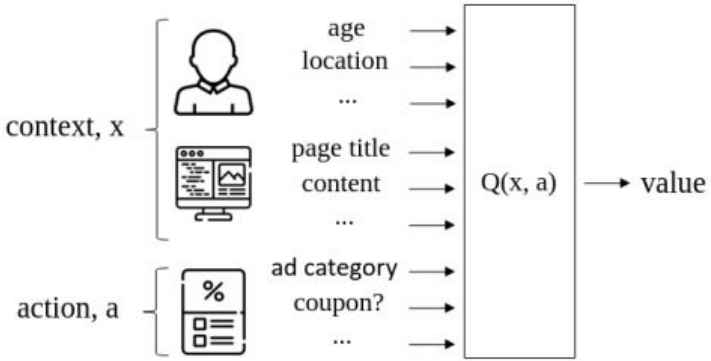
Case study – contextual online advertising with synthetic user data

Assume that the true user click behavior follows a logistic function:

$$p_a(x) = \frac{1}{1 + e^{-f_a(x)}}$$

$$f_a(x = [device, location, age]) = \beta_0^a + \beta_1^a \cdot device + \beta_2^a \cdot location + \beta_3^a \cdot age$$

Here, $p_a(x)$ is the probability of a user click when the context is x and ad a is shown. Also, let's assume that *device* is 1 for mobile and 0 otherwise; and *location* is 1 for US and 0 otherwise. There are two important things to note here:



Objective – regret minimization

A common metric that is used to compare MAB and CB algorithms is called **regret**. We define the total regret by the time we have observed the Kth user as follows:

$$\sum_{k=1}^{K} p_{a^*}(x_k) - p_a(x_k)$$

Here, x_k is the context for the kth user, a^* is the best action (ad) to take that gives the highest expected CTR, and a is the expected CTR for the selected action (ad). Note that we are able to calculate the regret because we have access to the true action values (expected CTRs), which would not be the case in reality (although regret can still be estimated). Note that the minimum possible regret at any step is zero.

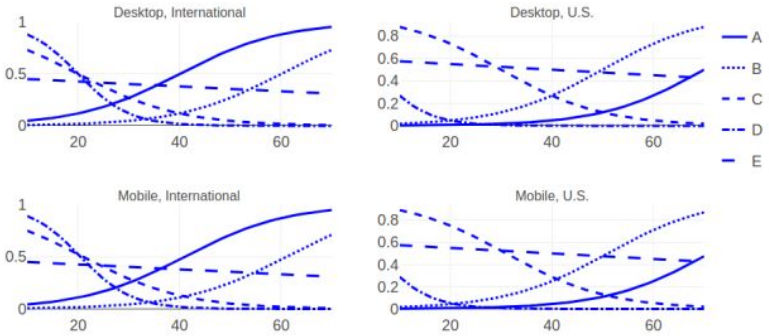


Figure 3.2 – Comparison of the true ad click probabilities given the context (x axis: age, y axis: click probability)

General Model of Linear Bandits

Assume the reward has the form:

$$r_t = x_t^\top \theta + \epsilon_t$$

where:

- x_t: feature vector of the selected action (arm) at time t
- θ: unknown parameter vector to be learned
- ϵ_t: noise term

Our goal is to estimate θ online from observed data.

LinearUCB

LinearUCB uses online ridge regression (least squares).

It maintains two quantities:

$$A_t = \lambda I + \sum_{s=1}^{t-1} x_s x_s^\top$$

$$b_t = \sum_{s=1}^{t-1} x_s r_s$$

Parameter estimate:

$$\hat{\theta}_t = A_t^{-1} b_t$$

This is exactly the solution of the optimization problem:

$$\min_\theta \sum_{s=1}^{t-1} (r_s - x_s^\top \theta)^2 + \lambda \|\theta\|^2$$

This is not SGD; it is a closed-form ridge regression solution, updated online.

Action score:

$$\text{score}(x) = x^\top \hat{\theta}_t + \alpha \sqrt{x^\top A_t^{-1} x}$$

Linear Thompson Sampling

LinearTS is the Bayesian version of linear regression.

Assume:

$$\theta \sim \mathcal{N}(0, \lambda^{-1} I)$$

$$r_t \mid x_t, \theta \sim \mathcal{N}(x_t^\top \theta, \sigma^2)$$

Posterior distribution:

$$\theta \mid D_t \sim \mathcal{N}(\hat{\theta}_t,\ \sigma^2 A_t^{-1})$$

where:

$$A_t = \lambda I + \sum_{s=1}^{t-1} x_s x_s^\top$$

$$\hat{\theta}_t = A_t^{-1} b_t$$

The update equations are the same as in LinearUCB. The difference is in action selection:

- LinearUCB uses an upper confidence bound.
- LinearTS samples

$$\tilde{\theta}_t \sim \mathcal{N}(\hat{\theta}_t,\ \sigma^2 A_t^{-1})$$

and chooses the action that maximizes $x^\top \tilde{\theta}_t$.

Contextual Bandit

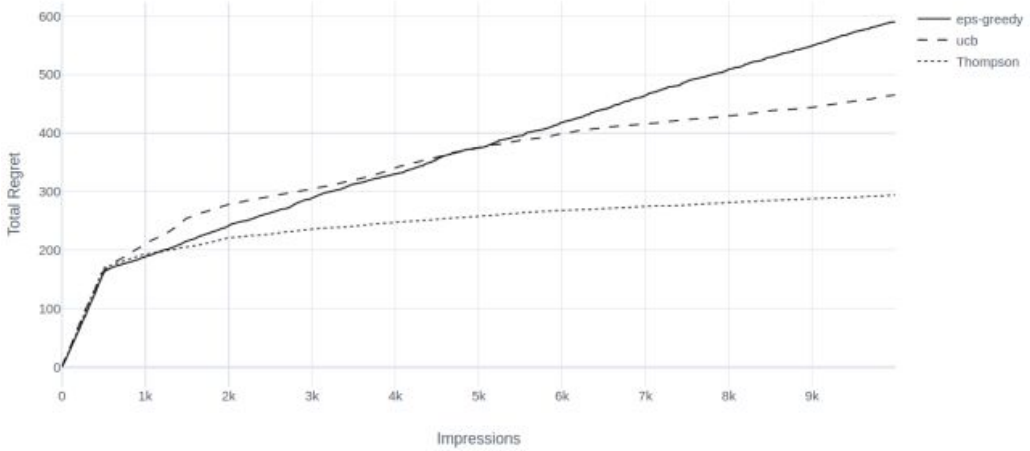


Figure 3.3 – Comparison of exploration strategies in the online advertising example

We clearly see that Thompson sampling is outperforming the ε-greedy and **upper confidence bound** (**UCB**) strategies as the total regret decelerates over time faster than the other two. It writes off the inefficient ads for given contexts pretty early on, while ε-greedy and UCB continue to explore those alternatives. Also note that we have not tuned the exploration rate for ε-greedy and `alpha` for UCB, which could have led to better performances. But this is exactly the point: Thompson sampling provides a very effective exploration strategy pretty much out of the box. This is what *Chapelle et al., 2011* empirically showed and helped the method gain popularity nearly a century after it was introduced.

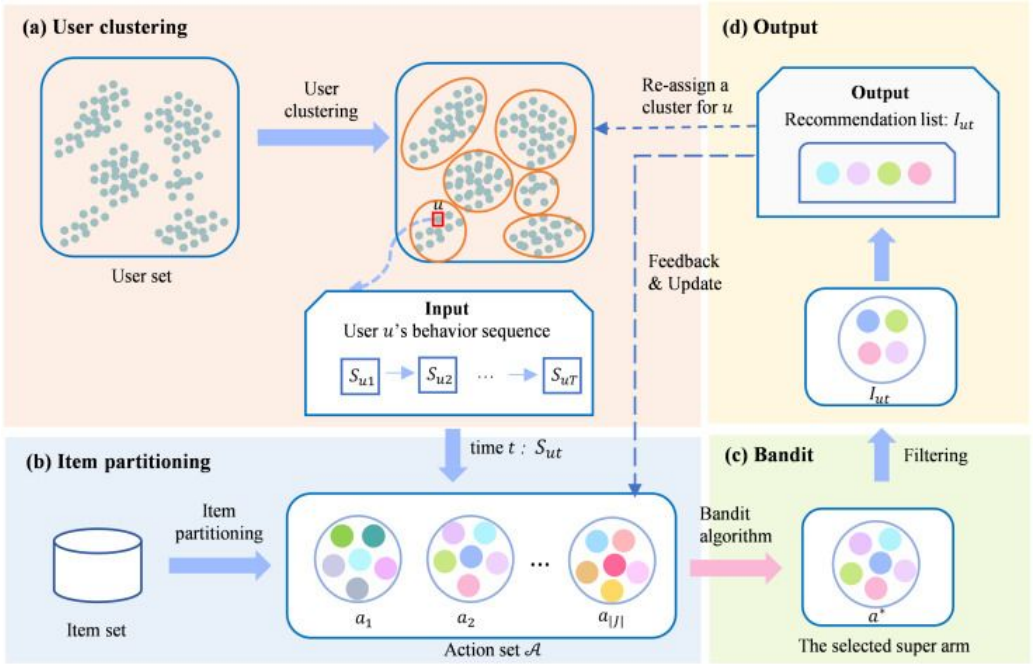
Clustered Bandit

Idea:

- Group users into clusters
- Run bandit per cluster

Motivation:

- Faster learning
- Share data among similar users



Source: Dynamic clustering based contextual combinatorial multi-armed bandit for online recommendation

User Segmentation

- Cluster users by:
  - Demographics
  - Behavior
  - Embeddings

Examples:

- New vs loyal users
- High spenders vs low spenders

Sharing Exploration

- Exploration is expensive
- Clusters allow:
  - Shared knowledge
  - Faster cold-start

Trade => bias vs efficiency

Use Case: New Item Exploration

Problem:

- New items have no clicks

Solution:

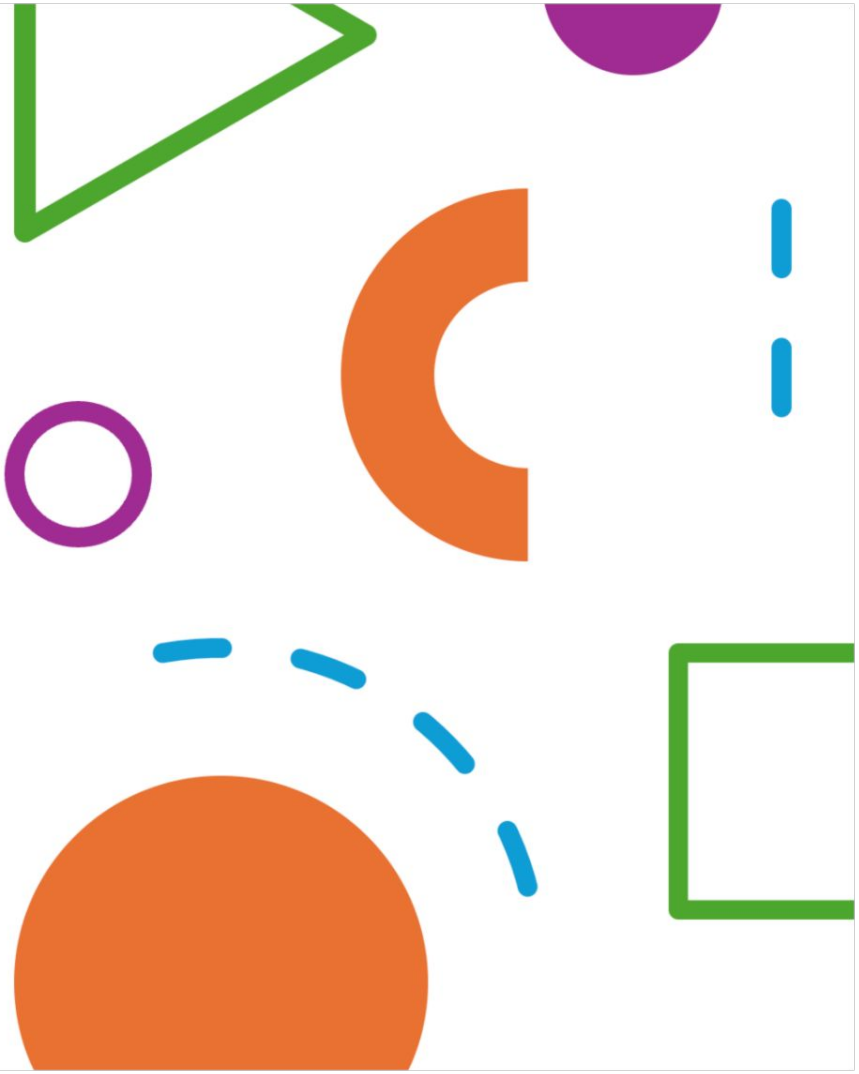
- Bandit allocates exposure
- Learns true CTR quickly

Use Case: Homepage Slot Optimization

- Which item goes to top slot?
- Reward = click / conversion
- Context = user features

Bandit chooses best item per slot

Session 7:
Sequential Modeling



Agenda overview:

- Why sequential behavior matters in recommendation
- How BERT4Rec works
- GRU4Rec, SASRec, and BERT4Rec
- What **Inductive Bias** is and how to mitigate it

Why Sequential RecSys ?

Traditional RecSys:

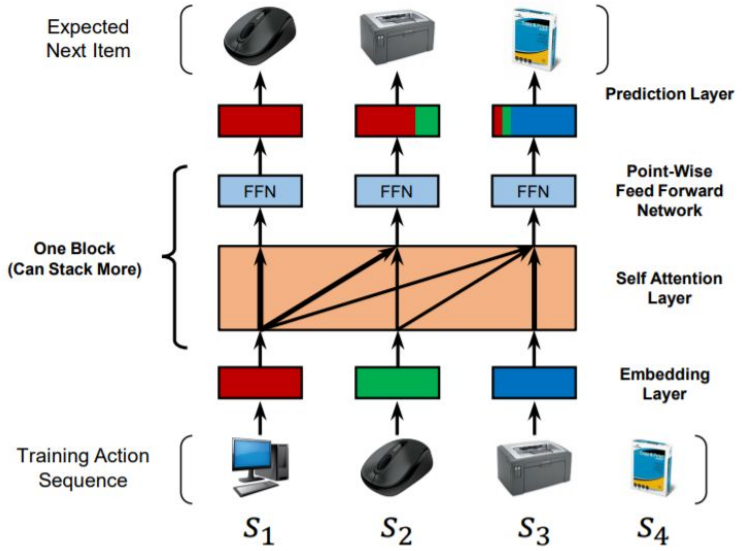
- User = static profile
- Predict next item from long-term preference

Sequential RecSys:

- User = evolving state
- Predict next item from **recent interactions**

Key assumption:

> Order matters



Session vs Long-term Behavior

Session-based behavior

- Short-term intent
- Examples: browsing session, shopping cart
- High volatility

Long-term behavior

- Stable preferences
- Examples: favorite genres, brands

Goal:

Combine both:

> "What does the user usually like?"
> "What does the user want *right now*?"

Problem Formulation

Given:

- User interaction sequence

  $(i_1, i_2, i_3, \ldots, i_t)$

Predict:

- Next item i_{t+1}

This is:

> Sequence modeling problem
> Similar to language modeling

Evolution:

- Markov Chains
- RNN-based models (GRU4Rec)
- Transformer-based models (SASRec, BERT4Rec)

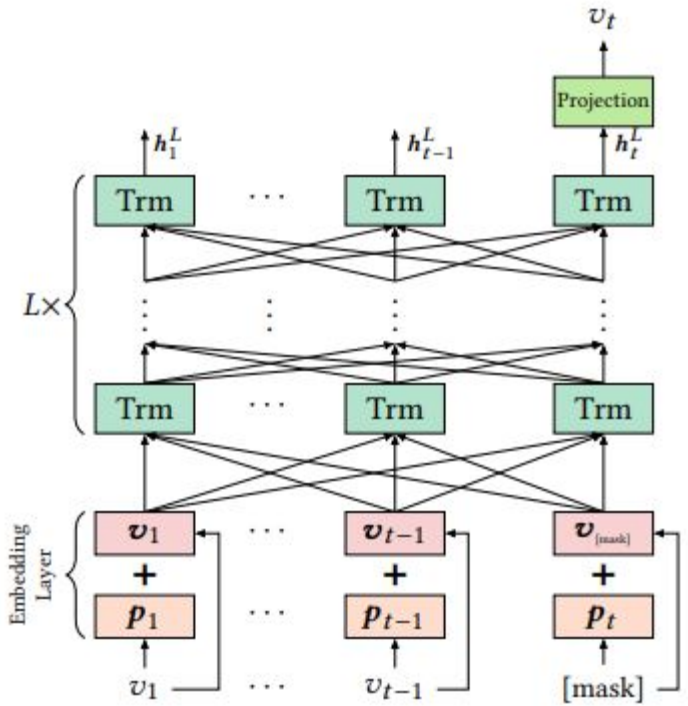
Transition to BERT4Rec

Key question:

Can we apply **BERT** to recommendation?

Differences vs NLP:

- Tokens = items
- Sentence = user sequence
- Word prediction = next-item prediction



Bert4Rec Training

Training:

- Randomly mask items
- Predict masked tokens
- Cross-entropy loss

Inference:

- Mask last item
- Predict next item

GRU4Rec vs. SASRec vs. Bert4rec

When to Use Which?

GRU4Rec:

- Short sessions
- Low compute

SASRec:

- Long sequences
- Online serving

BERT4Rec:

- Offline embedding learning
- High-accuracy batch recommendation

Model	Structure	Attention	Objective
GRU4Rec	RNN	Left only	Next item
SASRec	Transformer	Left only	Next item
BERT4Rec	Transformer	Bidirectional	Masked item

What is Inductive Bias ?

Inductive bias:

> Assumptions built into a model that guide learning

Examples:

- CNN assumes locality
- RNN assumes temporal order
- Transformer assumes global dependency

Inductive Bias in RecSys

Sequential models assume:

- Order matters
- Neighbor interactions dominate
- Next item depends on recent history

But reality:

- Multiple user intents
- Noisy behavior
- Exposure bias

Inductive Bias in Sequential Modeling

Limitation of self-attention in Sequential Recommendation (**SR**) systems: it is inherently **low-pass filtering in nature**, which leads to an **oversmoothing problem**.

- **Oversmoothing** occurs because Self-Attention averages out detailed patterns by focusing on all data, hindering the capture of crucial temporal dynamics.
- Most Transformer-based models use only low-pass filters, making them good at capturing **long-term, ongoing preferences** (e.g., being an "Apple fanatic") but poor at identifying **short-term, new interests or trends** (e.g., a sudden need for a "snorkel mask for vacation").
- Recommending items based on long-term interests is a simpler task, while accurately **predicting short-term interests** is a **significant challenge** for these models

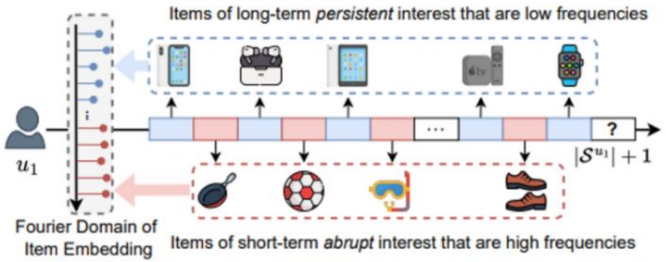


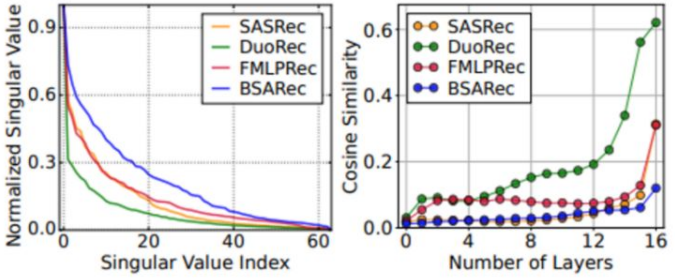
Figure 1: Illustration of high and low-frequency signals in SR. A user u_1's long-term persisting interests and tastes constitute low frequencies in the Fourier domain of embedding, and abrupt short-term changes in u_1's interests correspond to high frequencies.

Reference:

- Inductive Biases and Variable Creation in Self-Attention Mechanisms
- An Attentive Inductive Bias for Sequential Recommendation beyond the Self-Attention

Self-Attention is a Low-Pass Filter

- **The Oversmoothing Problem:** When self-attention is applied through many successive layers in deep Transformer models, it causes a loss of distinct feature expressiveness.
- **Loss of Representation:** Deep layers fail to maintain unique feature representations, degrading the model's ability to process information effectively.
- **Empirical Evidence:** As the number of layers increases, feature representations become overly uniform (indicated **by increased cosine similarity**), and **singular values decay rapidly**.
- **Performance Impact:** This uniformity prevents the model from capturing detailed preferences, leading inevitably to performance degradation.



(a) Singular value (b) Cosine similarity

Figure 3: Visualization of oversmoothing in LastFM. The singular values and cosine similarity of user sequence output embedding.

Reference:

- Inductive Biases and Variable Creation in Self-Attention Mechanisms
- Addressing the Rank Degeneration in Sequential Recommendation via Singular Spectrum Smoothing
- An Attentive Inductive Bias for Sequential Recommendation beyond the Self-Attention

Transformer-based Sequential Modeling for RecSys

Stacked Attention Layers in RecSys **lack the regularization** and **semantic richness** seen in NLP models:

- **User Sequences are typically short**, providing limited contextual information
- **Behavioral patterns are highly repetitive**, reducing the diversity of training signals
- **Interactions are semantically sparse**, with weak or noisy item-to-item relationships

These factors increase the risk of representation collapse

Reference:

- Inductive Biases and Variable Creation in Self-Attention Mechanisms
- Addressing the Rank Degeneration in Sequential Recommendation via Singular Spectrum Smoothing
- An Attentive Inductive Bias for Sequential Recommendation beyond the Self-Attention

Beyond Self-Attention for Sequential Recommendation (BSARec)

Beyond Self-Attention for Sequential Recommendation (BSARec):

- **BSARec** enhances standard self-attention by incorporating an **inductive bias** using a **Fourier transform**.
- The Fourier transform allows the model to access frequency information, which helps capture essential **patterns and periodicity** (such as **short-term trends** and **long-term interests**) that self-attention might miss.
- To address the issue of **oversmoothing** in Transformer-based models, BSARec introduces a **Frequency Rescaler** that acts as a **high-pass filter**, enabling the capture of high-frequency behavioral patterns.

Method	Inductive Bias	Self-Attention	High-pass Filter
SASRec	✗	✓	✗
BERT4Rec	✗	✓	✗
FMLPRec	✓	✗	✗
DuoRec	✗	✓	✗
BSARec	✓	✓	✓

Table 1: Comparison of existing Transformer-based methods that differ at three points: i) using inductive bias, ii) using self-attentions, and iii) using high-pass filters

Reference:

- Inductive Biases and Variable Creation in Self-Attention Mechanisms
- Filter-enhanced MLP is All You Need for Sequential Recommendation
- An Attentive Inductive Bias for Sequential Recommendation beyond the Self-Attention

Beyond Self-Attention for Sequential Recommendation (BSARec)

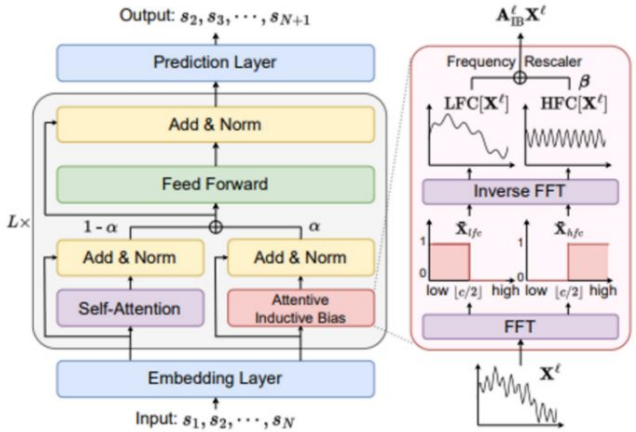


Figure 4: Architecture of our proposed BSARec. We propose a BSA encoder that uses both an inductive bias with a frequency rescaler and original self-attention.

Datasets	Metric	Caser	GRU4Rec	SASRec	BERT4Rec	FMLPRec	DuoRec	FEARec	BSARec	Improv.
Beauty	HR@5	0.0125	0.0169	0.0340	0.0469	0.0346	0.0707	0.0706	**0.0736**	4.10%
	HR@10	0.0225	0.0304	0.0531	0.0705	0.0559	0.0965	0.0982	**0.1008**	2.65%
	HR@20	0.0403	0.0527	0.0823	0.1073	0.0869	0.1313	0.1352	**0.1373**	1.55%
	NDCG@5	0.0076	0.0104	0.0221	0.0311	0.0222	0.0501	0.0512	**0.0523**	2.15%
	NDCG@10	0.0108	0.0147	0.0283	0.0387	0.0291	0.0584	0.0601	**0.0611**	1.66%
	NDCG@20	0.0153	0.0203	0.0356	0.0480	0.0369	0.0671	0.0694	**0.0703**	1.30%
Sports	HR@5	0.0091	0.0118	0.0188	0.0275	0.0220	0.0396	0.0411	**0.0426**	3.65%
	HR@10	0.0163	0.0187	0.0298	0.0428	0.0336	0.0569	0.0589	**0.0612**	3.90%
	HR@20	0.0260	0.0303	0.0459	0.0649	0.0525	0.0791	0.0836	**0.0858**	2.63%
	NDCG@5	0.0056	0.0079	0.0124	0.0180	0.0146	0.0276	0.0286	**0.0300**	4.90%
	NDCG@10	0.0080	0.0101	0.0159	0.0229	0.0183	0.0331	0.0343	**0.0360**	4.96%
	NDCG@20	0.0104	0.0131	0.0200	0.0284	0.0231	0.0387	0.0405	**0.0422**	4.20%
Toys	HR@5	0.0095	0.0121	0.0440	0.0412	0.0432	0.0770	0.0783	**0.0805**	2.81%
	HR@10	0.0161	0.0211	0.0652	0.0635	0.0671	0.1034	0.1054	**0.1081**	2.56%
	HR@20	0.0268	0.0348	0.0929	0.0939	0.0974	0.1369	0.1397	**0.1435**	2.72%
	NDCG@5	0.0058	0.0077	0.0297	0.0282	0.0288	0.0568	0.0574	**0.0589**	2.61%
	NDCG@10	0.0079	0.0106	0.0366	0.0353	0.0365	0.0653	0.0661	**0.0679**	2.72%
	NDCG@20	0.0106	0.0140	0.0435	0.0430	0.0441	0.0737	0.0747	**0.0768**	2.81%
Yelp	HR@5	0.0117	0.0130	0.0149	0.0256	0.0159	0.0271	0.0262	**0.0275**	1.48%
	HR@10	0.0197	0.0221	0.0249	0.0433	0.0287	0.0442	0.0437	**0.0465**	5.20%
	HR@20	0.0337	0.0383	0.0424	0.0717	0.0490	0.0717	0.0691	**0.0746**	4.04%
	NDCG@5	0.0070	0.0080	0.0091	0.0159	0.0100	**0.0170**	0.0165	**0.0170**	0.00%
	NDCG@10	0.0096	0.0109	0.0123	0.0216	0.0142	0.0225	0.0221	**0.0231**	2.67%
	NDCG@20	0.0131	0.0150	0.0167	0.0287	0.0192	0.0294	0.0285	**0.0302**	2.72%
LastFM	HR@5	0.0303	0.0312	0.0413	0.0294	0.0367	0.0431	0.0431	**0.0523**	21.35%
	HR@10	0.0431	0.0404	0.0633	0.0459	0.0560	0.0624	0.0587	**0.0807**	27.49%
	HR@20	0.0642	0.0541	0.0927	0.0596	0.0826	0.0963	0.0826	**0.1174**	21.91%
	NDCG@5	0.0227	0.0217	0.0284	0.0198	0.0243	0.0300	0.0304	**0.0344**	13.16%
	NDCG@10	0.0268	0.0245	0.0355	0.0252	0.0306	0.0361	0.0354	**0.0435**	20.50%
	NDCG@20	0.0321	0.0280	0.0429	0.0286	0.0372	0.0446	0.0414	**0.0526**	17.94%
ML-1M	HR@5	0.0927	0.1005	0.1374	0.1512	0.1316	0.1838	0.1834	**0.1944**	5.77%
	HR@10	0.1556	0.1657	0.2137	0.2346	0.2065	0.2704	0.2705	**0.2757**	1.92%
	HR@20	0.2488	0.2664	0.3245	0.3440	0.3137	0.3738	0.3714	**0.3884**	3.91%
	NDCG@5	0.0592	0.0619	0.0873	0.1021	0.0846	0.1252	0.1236	**0.1306**	4.31%
	NDCG@10	0.0795	0.0828	0.1116	0.1289	0.1087	0.1530	0.1516	**0.1568**	2.48%
	NDCG@20	0.1028	0.1081	0.1395	0.1564	0.1356	0.1790	0.1771	**0.1851**	3.41%

Table 2: Performance comparison of different methods on 6 datasets. The best results are in boldface and the second-best results are underlined. 'Improv.' indicates the relative improvement against the best baseline performance.

An Attentive Inductive Bias for Sequential Recommendation beyond the Self-Attention

Learnable Sequence Augmenter

- **Limitation of existing methods:** Most current contrastive learning-based sequential recommendation (SR) methods use random augmentations (cropping, reordering, substituting) to create positive sample pairs.
  - **Problem:** These random methods often fail to generate augmented sequences that accurately reflect the original sequence's representations, which can disrupt item correlations by removing key items or introducing noise.
- **Proposed solution:** A new method called Learnable sequence Augmentor for triplet Contrastive Learning in sequential Recommendation (LACLRec)
  - **How LACLRec works:** The method uses a self-supervised, learnable augmenter that automatically identifies and deletes noisy items while inserting new items that better capture item transition patterns, resulting in higher-quality augmented sequences

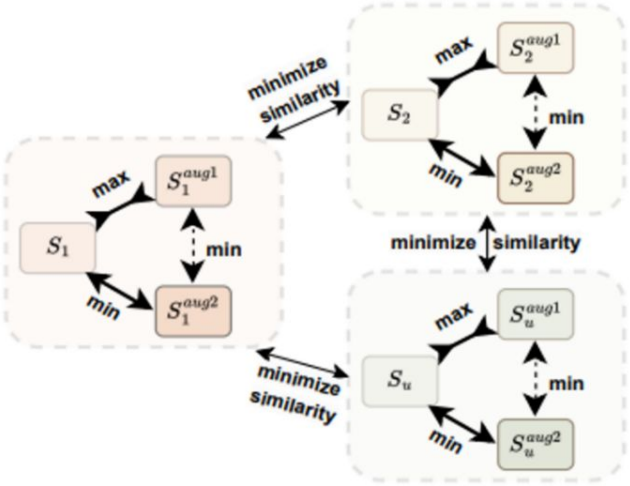


Figure 1. Illustration of triplet contrastive learning.

Learnable Sequence Augmenter for Triplet Contrastive Learning in Sequential Recommendation

Learnable Sequence Augmenter & Triplet Contrastive Learning

Step 1: Training an Augmenter: A sequence augmenter is trained to restore deliberately disrupted sequences (via random deletions/insertions). Once trained, this augmenter automatically produces "clean," SSL-augmented sequences without relying on manual rules.

Step 2: Contrastive Learning: The recommender uses a standard contrastive loss where a "positive pair" consists of one SSL-augmented sequence and one randomly augmented sequence from the same original source. Other sequences serve as negative samples to provide self-supervised signals.

Step 3: Finer-Grained Optimization: A "triplet" (raw sequence, SSL-augmented sequence, random augmented sequence) is used with a BPR contrastive loss. This loss maximizes similarity between the raw and the clean (SSL) sequence while minimizing similarity between the raw and the noisy (random) sequence.

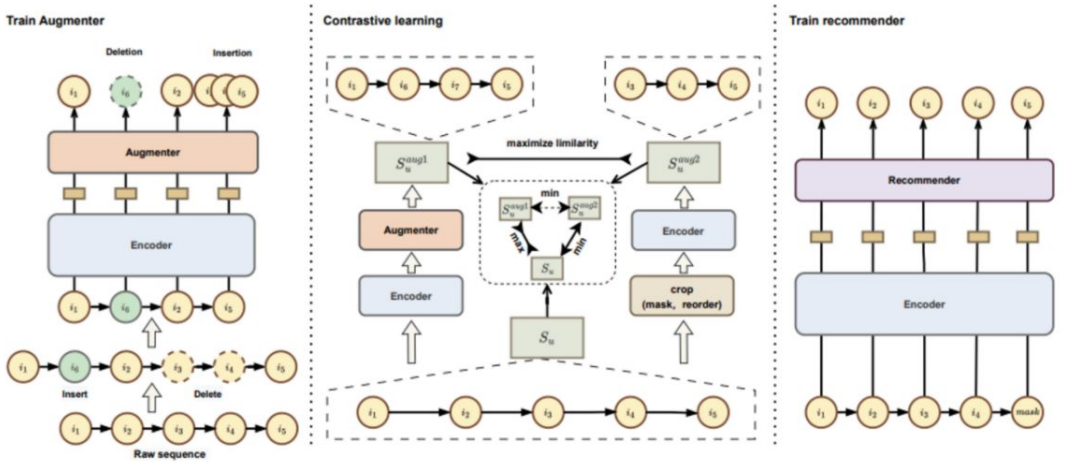


Figure 2. Overview of LACLRec. When training augmenter, we require it to restore the random modified sequences. The augmented sequence S_u^{aug1} is generated by the augmenter, and S_u^{aug2} is generated by random methods. Two contrastive learning tasks are used to provide supervised signals for item correlations learning. Finally, the recommender predicts the next item.

Learnable Sequence Augmenter for Triplet Contrastive Learning in Sequential Recommendation

Sequence Augmenter & Triplet Contrastive Learning

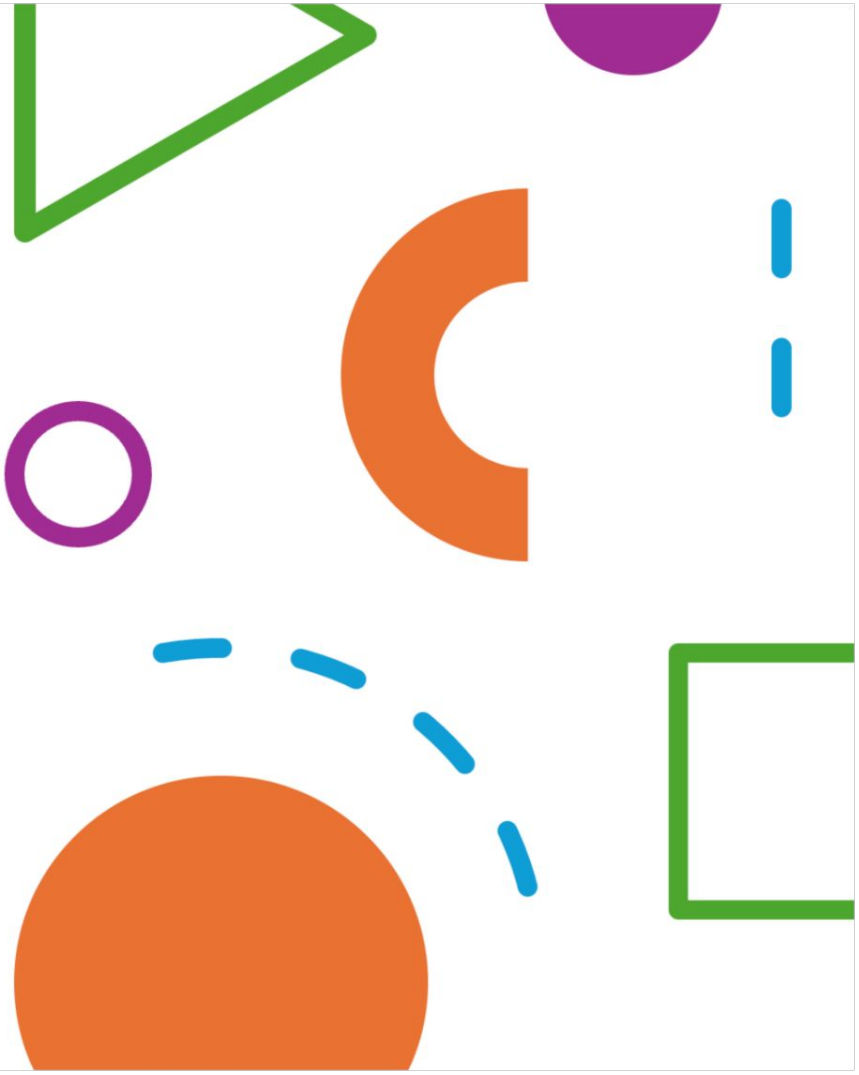
Table 2. Overall performance. The best performance and the second best performance are denoted in bold and underlined fonts respectively.

Dataset	Metrics	SASRec	BERT4Rec	CL4SRec	DuoRec	CoSeRec	ICSRec	STEAM	LACLRec	improve v.s. CL4SRec	improve v.s. All
Beauty	HR5	0.3721	0.3666	0.4067	0.4094	0.4124	0.4250	0.4284	**0.4446**	9.32%	3.78%
	HR10	0.4639	0.4728	0.5056	0.5078	0.5187	0.5188	0.5256	**0.5460**	7.99%	3.88%
	HR20	0.5804	0.6011	0.6199	0.6200	0.6356	0.6345	0.6486	**0.6606**	6.57%	1.85%
	MRR5	0.2611	0.2337	0.2785	0.2884	0.2749	0.2963	0.2899	**0.3161**	13.5%	6.68%
	MRR10	0.2732	0.2478	0.2916	0.3015	0.2890	0.3087	0.3028	**0.3297**	13.07%	6.8%
	MRR20	0.2812	0.2566	0.2994	0.3092	0.2971	0.3167	0.3112	**0.3376**	12.76%	6.6%
	NDCG5	0.2887	0.2570	0.3104	0.3194	0.3090	0.3284	0.3181	**0.3482**	12.18%	6.03%
	NDCG10	0.3182	0.2907	0.3422	0.3494	0.3434	0.3586	0.3508	**0.3810**	11.34%	6.25%
	NDCG20	0.3476	0.3227	0.3710	0.3774	0.3728	0.3878	0.3807	**0.4099**	10.49%	5.7%
	Sum	3.1868	3.049	3.4253	3.4825	3.4529	3.5748	3.5561	**3.7740**	10.18%	5.57%
Yelp	HR5	0.5847	0.6118	0.6292	0.6400	0.6454	0.6618	0.6683	**0.6833**	8.6%	2.24%
	HR10	0.7833	0.7972	0.8223	0.8263	0.8241	0.8337	0.8460	**0.8569**	4.21%	1.17%
	HR20	0.9194	0.9235	0.9486	0.9564	0.9444	0.9531	0.9511	**0.9554**	0.72%	0.24%
	MRR5	0.3543	0.3764	0.3991	0.4085	0.4140	0.4302	0.4293	**0.4482**	12.3%	4.18%
	MRR10	0.3808	0.4013	0.4251	0.4334	0.4380	0.4533	0.4531	**0.4716**	10.94%	4.04%
	MRR20	0.3907	0.4104	0.4343	0.4429	0.4467	0.4619	0.4607	**0.4788**	10.25%	3.66%
	NDCG5	0.4113	0.4257	0.4662	0.4647	0.4714	0.4877	0.4895	**0.5066**	11.05%	3.49%
	NDCG10	0.4756	0.4870	0.5188	0.5245	0.5293	0.5434	0.5461	**0.5630**	8.52%	3.09%
	NDCG20	0.5105	0.5218	0.5513	0.5590	0.5602	0.5741	0.5735	**0.5883**	6.71%	2.47%
	Sum	4.8109	4.9551	5.1849	5.2557	5.2734	5.3993	5.4176	**5.5523**	7.09%	2.49%
Sports	HR5	0.3450	0.3515	0.3935	0.3980	0.4074	0.4220	0.4190	**0.4379**	11.28%	3.77%
	HR10	0.4597	0.4791	0.5152	0.5192	0.5305	0.5459	0.5496	**0.5645**	9.57%	2.71%
	HR20	0.5942	0.6280	0.6562	0.6583	0.6650	0.6861	0.6916	**0.7017**	6.93%	1.46%
	MRR5	0.2204	0.2154	0.2600	0.2597	0.2653	0.2741	0.2685	**0.2896**	11.38%	5.65%
	MRR10	0.2357	0.2323	0.2762	0.2758	0.2817	0.2906	0.2858	**0.3064**	10.93%	5.44%
	MRR20	0.2449	0.2426	0.2859	0.2854	0.2910	0.3003	0.2956	**0.3159**	10.49%	5.19%
	NDCG5	0.2513	0.2448	0.2931	0.2936	0.3005	0.3108	0.3058	**0.3264**	11.36%	5.02%
	NDCG10	0.2883	0.2862	0.3324	0.3335	0.3403	0.3508	0.3479	**0.3673**	10.5%	4.7%
	NDCG20	0.3222	0.3236	0.3680	0.3686	0.3743	0.3862	0.3838	**0.4020**	9.24%	4.09%
	Sum	2.9622	3.0035	0.3805	3.3921	3.4559	3.5668	3.5476	**3.7122**	9.81%	4.08%

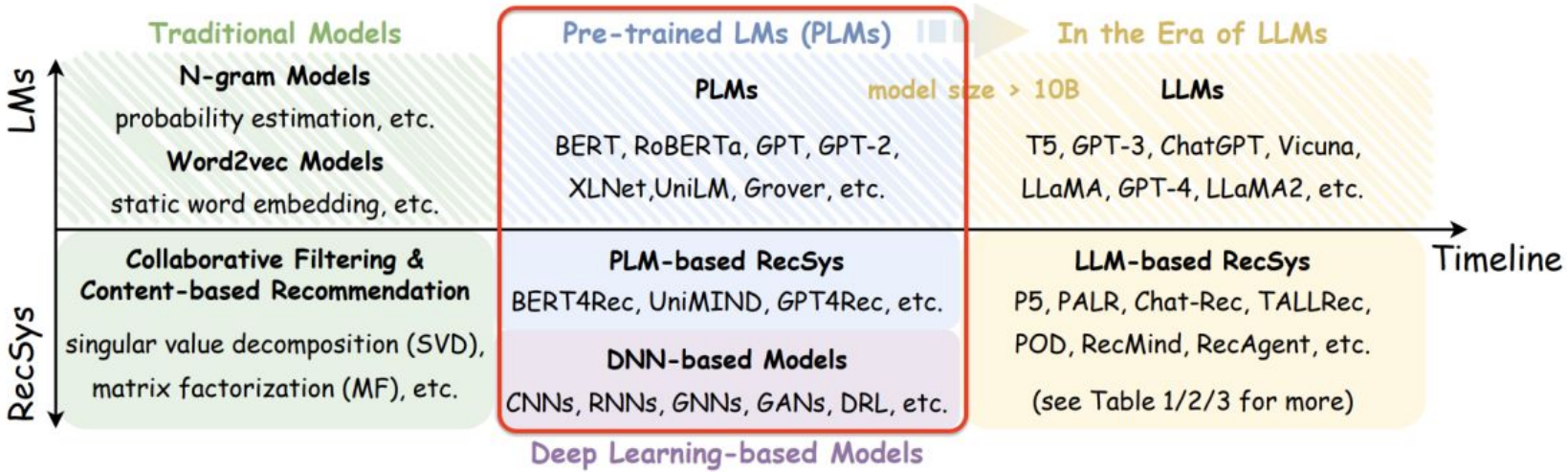
Learnable Sequence Augmenter for Triplet Contrastive Learning in Sequential Recommendation

Session 8:

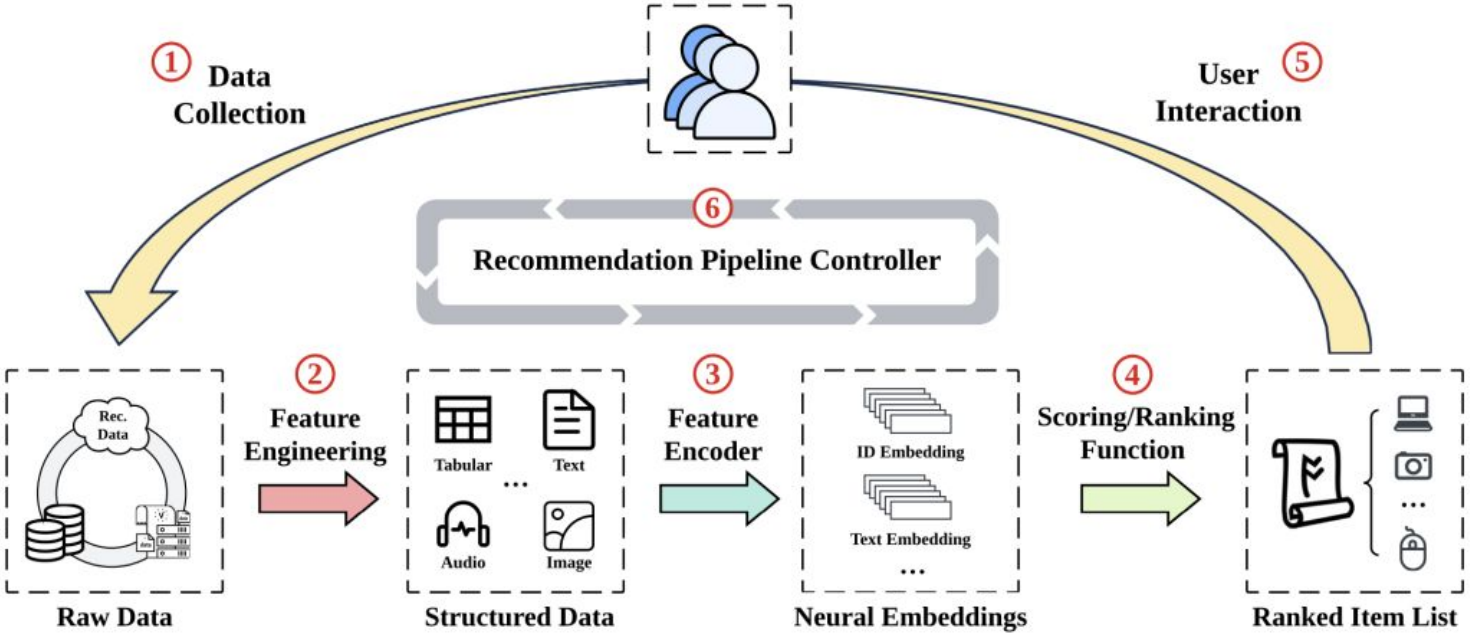
LLMs for RecSys



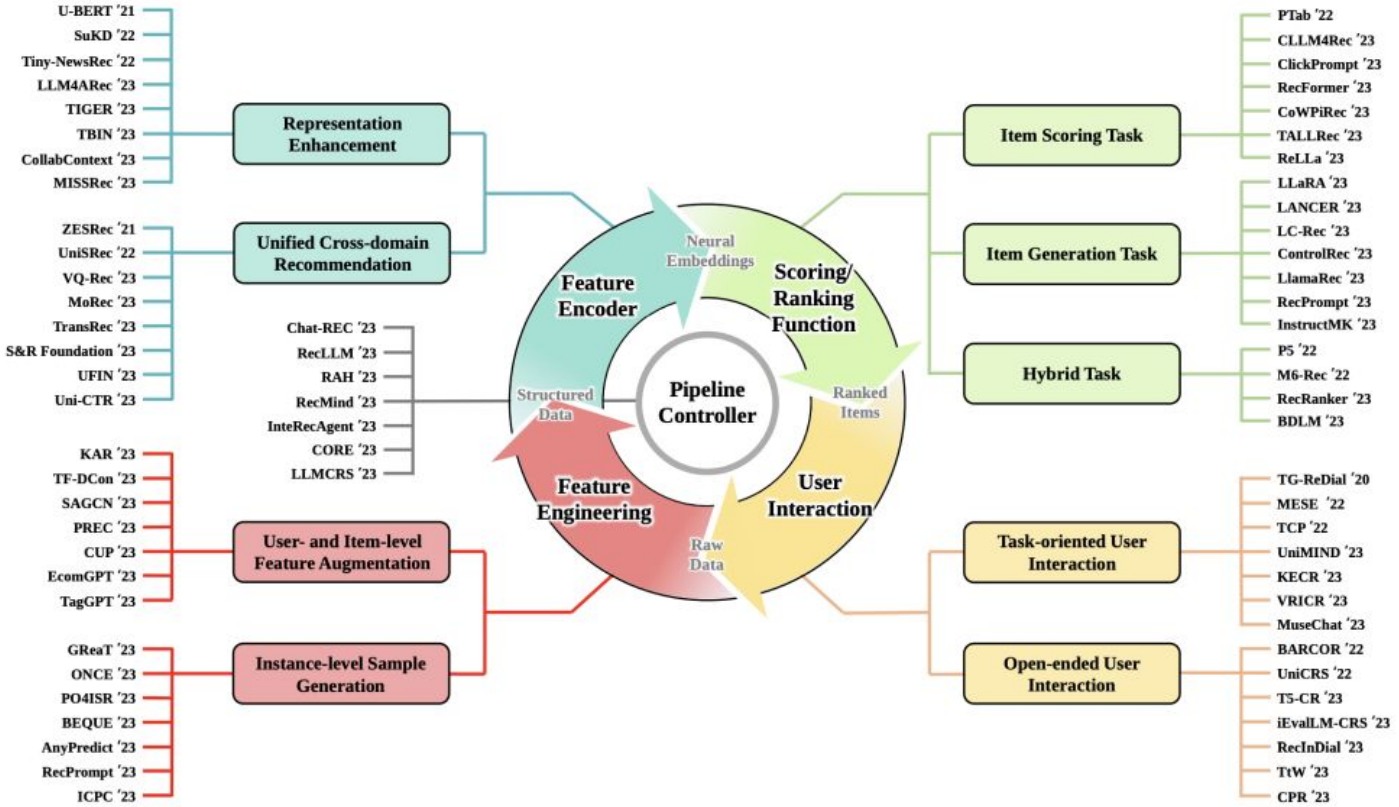
Evolution of LLMs & RecSys



A typical Deep Learning based Recommender System pipeline

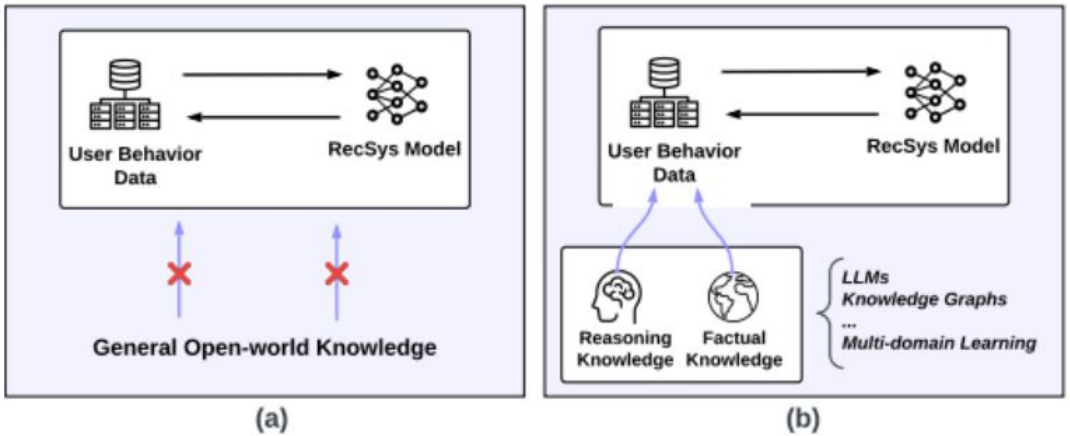


Topics of LLM Usage for RecSys



1. LLM for Feature Engineering – User and Item level Feature Augmentation

- LLM is strong at encoding extensive **world knowledge** and demonstrated some level of **reasoning capability**
- **KAR** adopts LLM to generate the **user-side preference knowledge** (based on **reasoning**) and **item-side factual knowledge**

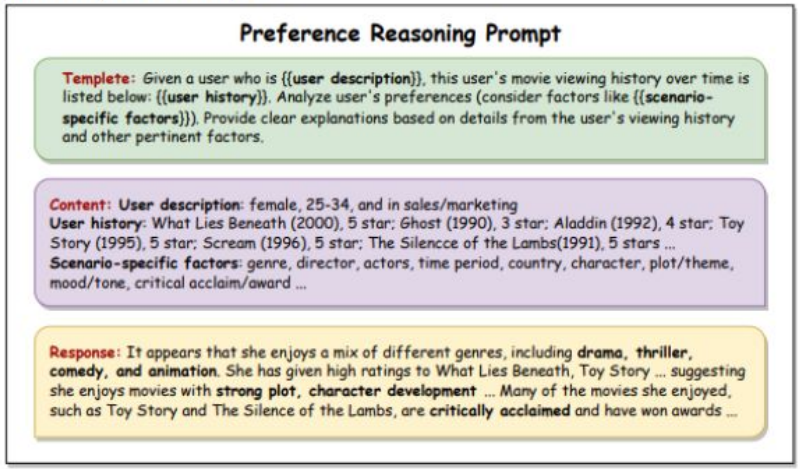


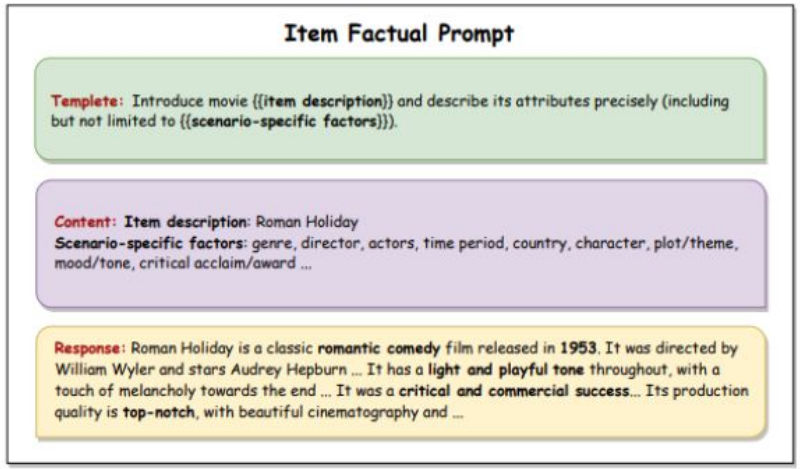
(a) closed recommender systems
and (b) open-world recommender systems

Towards Open-World Recommendation with Knowledge Augmentation from Large Language Models (KAR) - 2023

1. LLM for Feature Engineering – User and Item level Feature Augmentation

- LLM is strong at encoding extensive **world knowledge** and demonstrated some level of **reasoning capability**
- **KAR** adopts LLM to generate the **user-side preference knowledge** (based on **reasoning**) and **item-side factual knowledge**

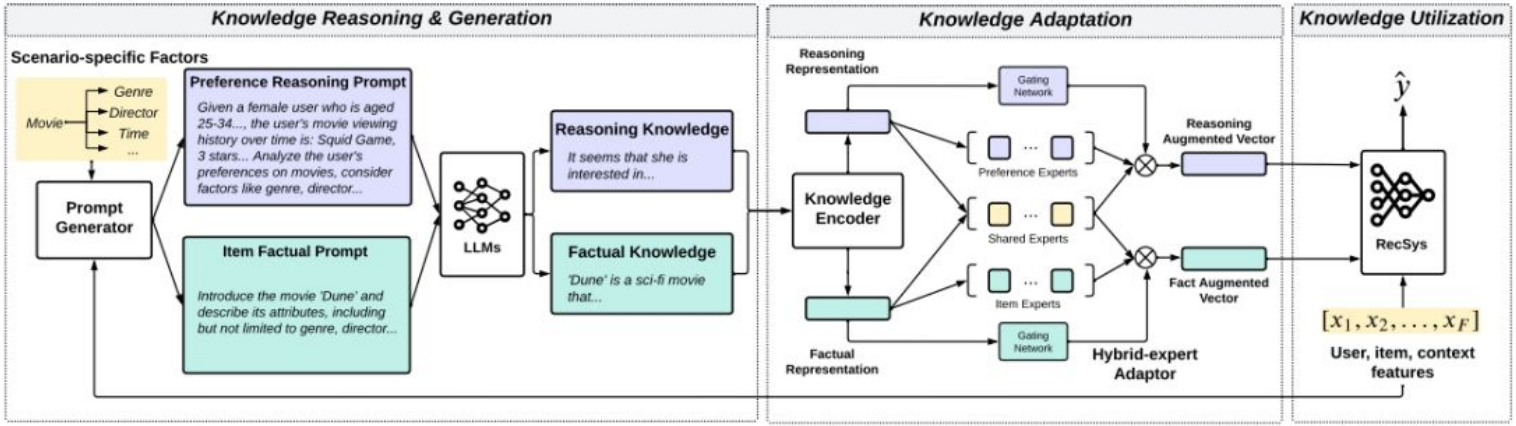




Towards Open-World Recommendation with Knowledge Augmentation from Large Language Models (KAR) - 2023

1. LLM for Feature Engineering – User and Item level Feature Augmentation

- The generated **reasoning** and **factual** **knowledge** are transformed and condensed into **augmented vectors** by a **hybrid-expert adaptor**



Towards Open-World Recommendation with Knowledge Augmentation from Large Language Models (KAR) - 2023

1. LLM for Feature Engineering – User and Item level Feature Augmentation

Backbone model	MovieLens-1M						Amazon-Books					
	AUC			LogLoss			AUC			LogLoss		
	base	KAR	Improv.	base	KAR	Improv.	base	KAR	Improv.	base	KAR	Improv.
DCNv2	0.7830	**0.7935***	1.34%	0.5516	**0.5410***	1.92%	0.8269	**0.8350***	0.98%	0.4973	**0.4865***	2.17%
DCNv1	0.7828	**0.7927***	1.28%	0.5528	**0.5411***	2.12%	0.8268	**0.8348***	0.97%	0.4973	**0.4869***	2.11%
DeepFM	0.7824	**0.7919***	1.22%	0.5518	**0.5432***	1.56%	0.8269	**0.8347***	0.94%	0.4969	**0.4873***	1.93%
FiBiNet	0.7820	**0.7936***	1.49%	0.5531	**0.5405***	2.27%	0.8269	**0.8351***	0.99%	0.4973	**0.4870***	2.07%
AutoInt	0.7821	**0.7931***	1.40%	0.5520	**0.5430***	1.62%	0.8262	**0.8357***	1.16%	0.4981	**0.4863***	2.37%
FiGNN	0.7832	**0.7935***	1.32%	0.5510	**0.5437***	1.33%	0.8270	**0.8352***	0.99%	0.4977	**0.4870***	2.14%
xDeepFM	0.7823	**0.7926***	1.32%	0.5520	**0.5420***	1.81%	0.8271	**0.8351***	0.97%	0.4971	**0.4866***	2.10%
DIEN	0.7853	**0.7953***	1.27%	0.5494	**0.5394***	1.83%	0.8307	**0.8391***	1.01%	0.4926	**0.4812***	2.32%
DIN	0.7863	**0.7961***	1.24%	0.5486	**0.5370***	2.10%	0.8304	**0.8418***	1.38%	0.4937	**0.4801***	2.77%

* denotes statistically significant improvement over backbone CTR prediction models (t-test with p-value < 0.05).

Read more:

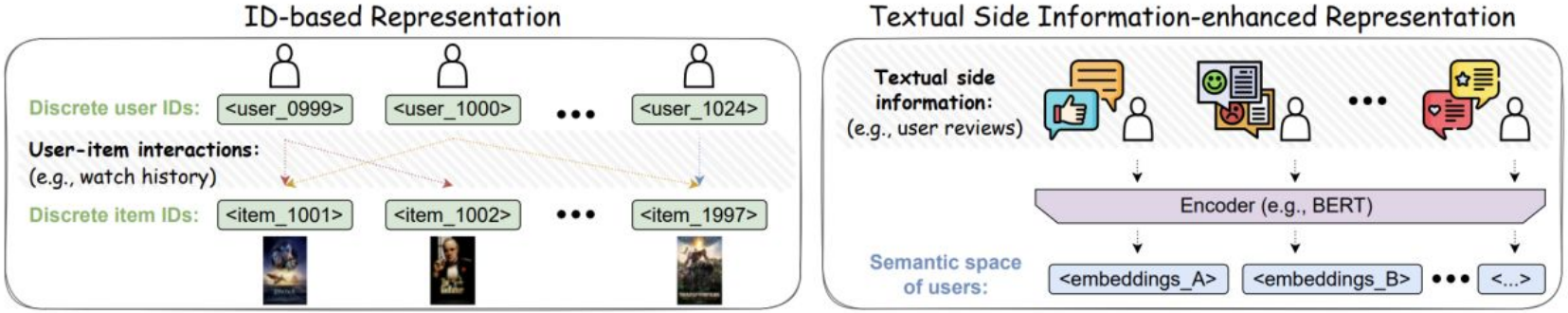
- SAGCN (2023) : use chain-based prompting to uncover semantic aspect-aware interactions, providing clearer insights into user behaviors at a fine-grained semantic level
- CUP (2023) : adopts ChatGPT to summarize each user's interests with a few short keywords according to the user review texts
- …

Instance-level Sample Generation (synthetic data generation) : read more in survey

2. LLM for Feature Encoder

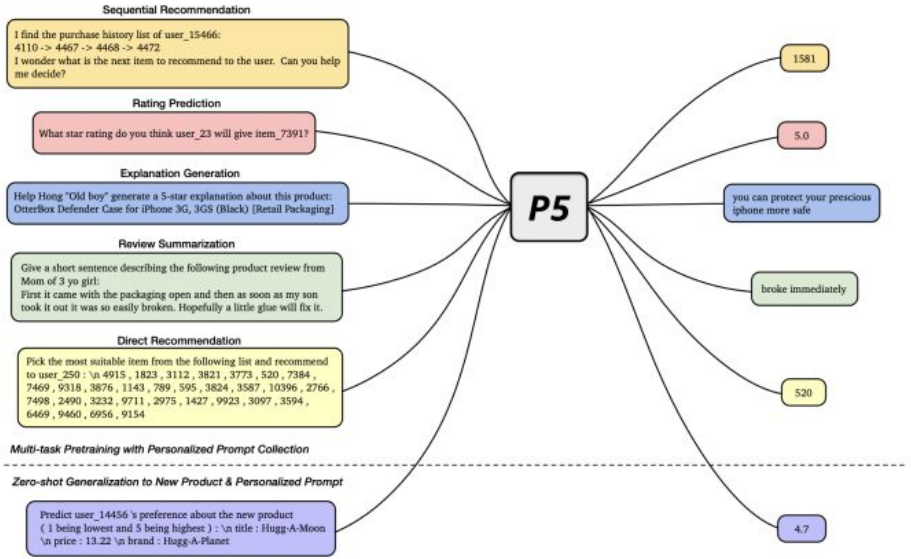
Two Methods for representing users and items in LLM-based RecSys:

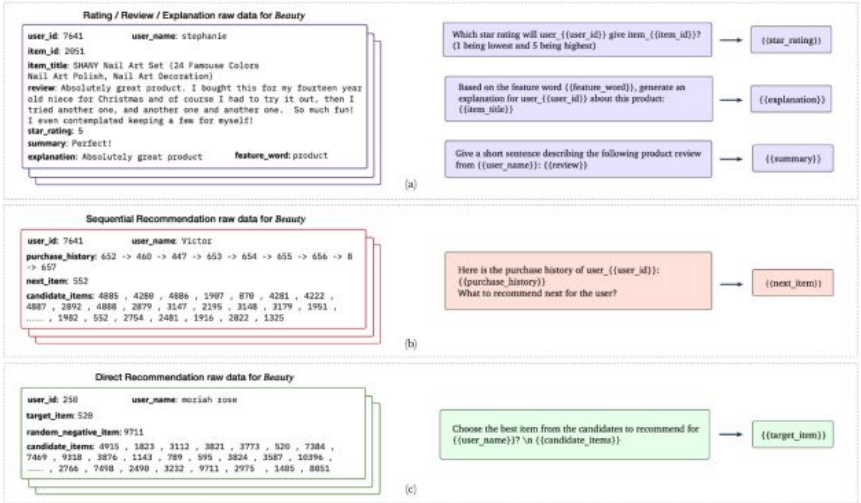
- ID-based Representation
- Textual Side Information-enhanced Representation



2. LLM for Feature Encoder - ID-based Representation (P5)

- User-item interactions, user descriptions, item metadata, and user reviews are converted to natural language sequence
- Adaptive personalized prompt => make predictions with **zero-shot** or **few-shot** (don't need fine-tuning)





Recommendation as Language Processing (RLP):
A Unified **P**retrain, **P**ersonalized **P**rompt & **P**redict **P**aradigm (P5) - 2022

2. LLM for Feature Encoder - ID-based Representation (P5)

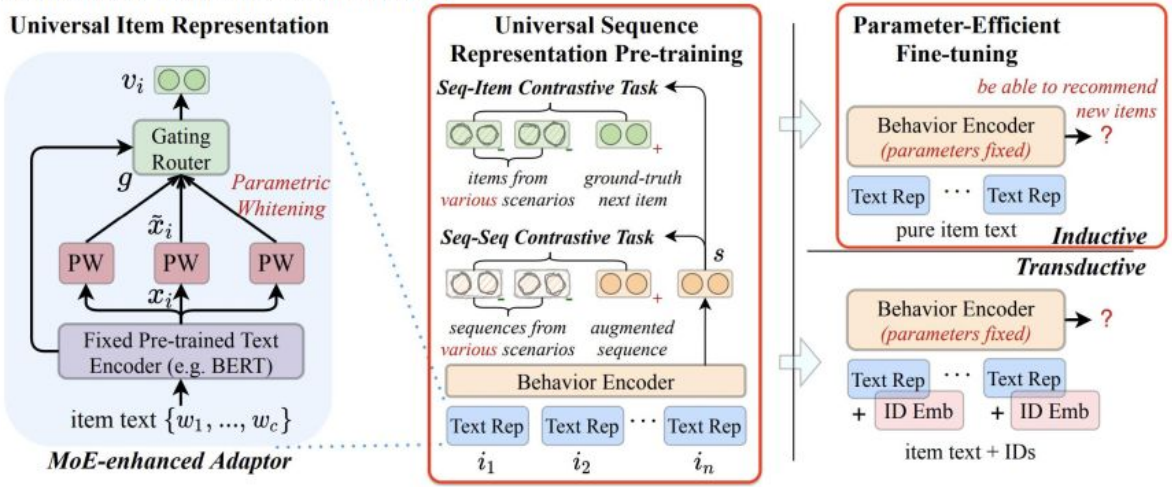
Methods	Sports				Beauty				Toys			
	HR@5	NDCG@5	HR@10	NDCG@10	HR@5	NDCG@5	HR@10	NDCG@10	HR@5	NDCG@5	HR@10	NDCG@10
Caser	0.0116	0.0072	0.0194	0.0097	0.0205	0.0131	0.0347	0.0176	0.0166	0.0107	0.0270	0.0141
HGN	0.0189	0.0120	0.0313	0.0159	0.0325	0.0206	0.0512	0.0266	0.0321	0.0221	0.0497	0.0277
GRU4Rec	0.0129	0.0086	0.0204	0.0110	0.0164	0.0099	0.0283	0.0137	0.0097	0.0059	0.0176	0.0084
BERT4Rec	0.0115	0.0075	0.0191	0.0099	0.0203	0.0124	0.0347	0.0170	0.0116	0.0071	0.0203	0.0099
FDSA	0.0182	0.0122	0.0288	0.0156	0.0267	0.0163	0.0407	0.0208	0.0228	0.0140	0.0381	0.0189
SASRec	0.0233	0.0154	0.0350	0.0192	0.0387	0.0249	0.0605	0.0318	0.0463	0.0306	0.0675	0.0374
S^3-Rec	0.0251	0.0161	0.0385	0.0204	0.0387	0.0244	0.0647	0.0327	0.0443	0.0294	0.0700	0.0376
P5-S (2-3)	0.0272	0.0169	0.0361	0.0198	0.0503	0.0370	0.0659	0.0421	**0.0648**	**0.0567**	**0.0709**	**0.0587**
P5-B (2-3)	0.0364	0.0296	0.0431	0.0318	**0.0508**	**0.0379**	**0.0664**	**0.0429**	0.0608	0.0507	0.0688	0.0534
P5-S (2-13)	0.0258	0.0159	0.0346	0.0188	0.0490	0.0358	0.0646	0.0409	0.0647	0.0566	0.0705	0.0585
P5-B (2-13)	**0.0387**	**0.0312**	**0.0460**	**0.0336**	0.0493	0.0367	0.0645	0.0416	0.0587	0.0486	0.0675	0.0536

Read more:

- How to Index Item IDs for Recommendation Foundation Models (2023)
- Recommender Systems with Generative Retrieval (TIGER) (2023)

2. LLM for Feature Encoder - Textual Side Information-enhanced Representation

- **Pure ID features** that inherently contains no semantic information
- **ID embedding** is **transductive**. Limit the transferability of item representation across domain & can not encode new item
- Use **BERT** to encode **item titles** or **descriptions**
- Simply using item desc emb will not work due to ***seesaw phenomenon***
- Key: **Seq-Item Contrastive** & **Seq-Seq Contrastive Learning**



Towards Universal Sequence Representation Learning for Recommender Systems (UniSRec) - 2022

2. LLM for Feature Encoder - Textual Side Information-enhanced Representation

Scenario	Dataset	Metric	SASRec	BERT4Rec	FDSA	S^3-Rec	CCDR	RecGURU	ZESRec	UniSRec$_t$	UniSRec$_{t+ID}$	Improv.
Cross-Domain	Scientific	Recall@10	0.1080	0.0488	0.0899	0.0525	0.0695	0.1023	0.0851	0.1188*	**0.1235***	+14.35%
		NDCG@10	0.0553	0.0243	0.0580	0.0275	0.0340	0.0572	0.0475	**0.0641***	0.0634*	+10.52%
		Recall@50	0.2042	0.1185	0.1732	0.1418	0.1647	0.2022	0.1746	0.2394*	**0.2473***	+21.11%
		NDCG@50	0.0760	0.0393	0.0759	0.0468	0.0546	0.0786	0.0670	0.0903*	**0.0904***	+15.01%
	Pantry	Recall@10	0.0501	0.0308	0.0395	0.0444	0.0408	0.0469	0.0454	0.0636*	**0.0693***	+38.32%
		NDCG@10	0.0218	0.0152	0.0209	0.0214	0.0203	0.0209	0.0230	0.0306*	**0.0311***	+35.22%
		Recall@50	0.1322	0.1030	0.1151	0.1315	0.1262	0.1269	0.1141	0.1658*	**0.1827***	+38.20%
		NDCG@50	0.0394	0.0305	0.0370	0.0400	0.0385	0.0379	0.0378	0.0527*	**0.0556***	+39.00%
	Instruments	Recall@10	0.1118	0.0813	0.1070	0.1056	0.0848	0.1113	0.0783	0.1189*	**0.1267***	+13.33%
		NDCG@10	0.0612	0.0620	**0.0796**	0.0713	0.0451	0.0681	0.0497	0.0680	0.0748*	–
		Recall@50	0.2106	0.1454	0.1890	0.1927	0.1753	0.2068	0.1387	0.2255*	**0.2387***	+13.34%
		NDCG@50	0.0826	0.0756	0.0972	0.0901	0.0647	0.0887	0.0627	0.0912	**0.0991***	+1.95%
	Arts	Recall@10	0.1108	0.0722	0.1002	0.1003	0.0671	0.1084	0.0664	0.1066	**0.1239***	+11.82%
		NDCG@10	0.0587	0.0479	**0.0714**	0.0601	0.0348	0.0651	0.0375	0.0586	0.0712	–
		Recall@50	0.2030	0.1367	0.1779	0.1888	0.1478	0.1979	0.1323	0.2049*	**0.2347***	+15.62%
		NDCG@50	0.0788	0.0619	0.0883	0.0793	0.0523	0.0845	0.0518	0.0799	**0.0955***	+8.15%
	Office	Recall@10	0.1056	0.0825	0.1118	0.1030	0.0549	0.1145	0.0641	0.1013	**0.1280***	+11.79%
		NDCG@10	0.0710	0.0634	**0.0868**	0.0653	0.0290	0.0768	0.0391	0.0619	0.0831	–
		Recall@50	0.1627	0.1227	0.1665	0.1613	0.1095	0.1757	0.1113	0.1702	**0.2016***	+14.74%
		NDCG@50	0.0835	0.0721	0.0987	0.0780	0.0409	0.0901	0.0493	0.0769	**0.0991**	+0.41%
Cross-Platform	Online Retail	Recall@10	0.1460	0.1349	0.1490	0.1418	0.1347	0.1467	0.1103	0.1449	**0.1537***	+3.15%
		NDCG@10	0.0675	0.0653	0.0719	0.0654	0.0620	0.0658	0.0535	0.0677	**0.0724**	+0.70%
		Recall@50	0.3872	0.3540	0.3802	0.3702	0.3587	**0.3885**	0.2750	0.3604	**0.3885**	0.00%
		NDCG@50	0.1201	0.1131	0.1223	0.1154	0.1108	0.1188	0.0896	0.1149	**0.1239***	+1.31%

Read more:

- One4all user representation for recommender systems in e-commerce (ShopperBERT) (2021)
- Where to go next for recommender systems? id-vs. modality-based recommender models revisited (2023)
- Exploring the upper limits of text-based collaborative filtering using large language models: Discoveries and insights (2023)
- Learning vector-quantized item representation for transferable sequential recommenders (VQ-Rec) (2023)

3. LLM for Scoring / Ranking

Large Language Model: masked LM or next token prediction task (~ **Retrieval** in RecSys)

$$h = \mathrm{LLM}(x),$$

$$s = \mathrm{LM_Head}(h) \in \mathbb{R}^V,$$

$$p = \mathrm{Softmax}(s) \in \mathbb{R}^V,$$

$$\hat{t} \sim p,$$

where h is the final representation, V is the vocabulary size, and $t\hat{}$ is the predicted token sampled from the probability distribution p.

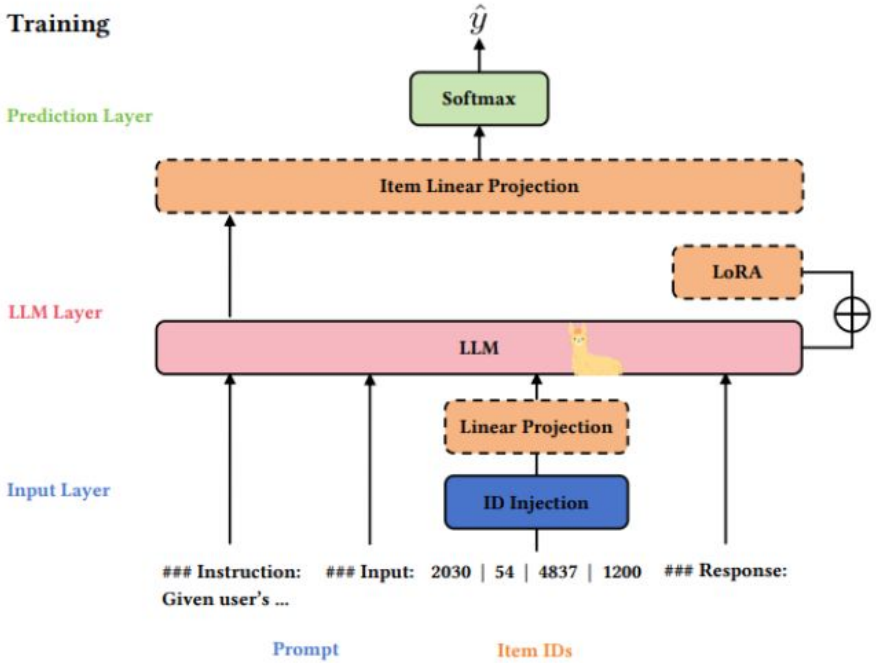
Item Scoring / Ranking Task (RecSys) requires the model to do **pointwise scoring** for a given **user-item pair (u,i)**, and the output should be a **real number** $y\hat{} = F(u,i)$, instead of **generated discrete tokens** $t\hat{}$

3. LLM for Scoring / Ranking

Two Methods for Item Scoring LLM-based:

- **Finetune last projection layer**
  - Single-Tower : $\hat{y} = F(u, i) = \mathrm{MLP}(h)$
  - Two-Tower : $\hat{y} = F(u, i) = d\left(T_u\left(x_u\right), T_i\left(x_i\right)\right)$
- Cast Item Scoring task as **Binary Question Answering** problem

3. LLM for Scoring / Ranking – Finetune last projection layer (Single-Tower)



E4SRec: An Elegant Effective Efficient Extensible Solution of Large Language Models for Sequential Recommendation - 2023

3. LLM for Scoring / Ranking – Finetune last projection layer (Single-Tower)

Dataset	Metric	POP	BPR	GRU4Rec	Caser	SASRec	BERT4Rec	S^3-Rec	CL4SRec	ICLRec	**E4SRec**	Improv.
Beauty	HR@5	0.0072	0.0120	0.0164	0.0251	0.0333	0.0193	0.0327	0.0407	0.0436	**0.0525**	20.41%
	HR@10	0.0114	0.0361	0.0289	0.0418	0.0581	0.0401	0.0591	0.0626	0.0653	**0.0758**	16.08%
	HR@20	0.0195	0.0589	0.0478	0.0643	0.0915	0.0596	0.0898	0.0957	0.0974	**0.1071**	9.96%
	nDCG@5	0.0040	0.0065	0.0086	0.0127	0.0179	0.0187	0.0175	0.0223	0.0240	**0.0360**	50.00%
	nDCG@10	0.0053	0.0122	0.0142	0.0193	0.0258	0.0254	0.0268	0.0317	0.0338	**0.0435**	28.70%
	nDCG@20	0.0073	0.0179	0.0169	0.0258	0.0342	0.0361	0.0370	0.0396	0.0416	**0.0514**	23.56%
Sports	HR@5	0.0055	0.0092	0.0137	0.0139	0.0170	0.0176	0.0157	0.0217	0.0238	**0.0281**	18.07%
	HR@10	0.0090	0.0188	0.0274	0.0231	0.0289	0.0326	0.0265	0.0374	0.0393	**0.0410**	4.33%
	HR@20	0.0149	0.0258	0.0438	0.0389	0.0477	0.0493	0.0460	0.0582	0.0553	**0.0626**	7.56%
	nDCG@5	0.0040	0.0053	0.0096	0.0085	0.0091	0.0105	0.0098	0.0129	0.0152	**0.0196**	28.95%
	nDCG@10	0.0051	0.0083	0.0137	0.0126	0.0129	0.0153	0.0135	0.0184	0.0212	**0.0237**	11.79%
	nDCG@20	0.0066	0.0121	0.0171	0.0166	0.0177	0.0195	0.0182	0.0239	0.0250	**0.0291**	16.40%
Toys	HR@5	0.0064	0.0120	0.0097	0.0166	0.0445	0.0274	0.0492	0.0484	0.0509	**0.0566**	11.20%
	HR@10	0.0079	0.0211	0.0196	0.0281	0.0698	0.0460	0.0698	0.0706	0.0725	**0.0798**	10.07%
	HR@20	0.0108	0.0312	0.0301	0.0420	0.0999	0.0688	0.0962	0.0984	0.1018	**0.1107**	8.74%
	nDCG@5	0.0037	0.0082	0.0059	0.0107	0.0236	0.0174	0.0342	0.0327	0.0350	**0.0405**	15.71%
	nDCG@10	0.0057	0.0120	0.0098	0.0151	0.0318	0.0230	0.0375	0.0404	0.0423	**0.0479**	13.24%
	nDCG@20	0.0062	0.0136	0.0116	0.0179	0.0394	0.0291	0.0431	0.0466	0.0493	**0.0557**	12.98%
Yelp	HR@5	0.0056	0.0127	0.0152	0.0142	0.0161	0.0186	0.0173	0.0216	0.0240	**0.0266**	10.83%
	HR@10	0.0083	0.0245	0.0263	0.0252	0.0265	0.0291	0.0282	0.0352	0.0381	**0.0418**	9.71%
	HR@20	0.0120	0.0346	0.0371	0.0406	0.0443	0.0564	0.0538	0.0585	0.0630	**0.0675**	7.14%
	nDCG@5	0.0036	0.0076	0.0104	0.0096	0.0102	0.0121	0.0114	0.0130	0.0150	**0.0189**	26.00%
	nDCG@10	0.0043	0.0119	0.0137	0.0129	0.0134	0.0171	0.0163	0.0185	0.0203	**0.0238**	17.24%
	nDCG@20	0.0056	0.0143	0.0145	0.0156	0.0179	0.0223	0.0201	0.0235	0.0256	**0.0297**	16.02%

Read more **Two-Tower Scoring**:

- One Model for All: Large Language Models are Domain-Agnostic Recommendation Systems (LLM-Rec) - 2023

3. LLM for Scoring / Ranking as Binary Question Answering Task

- PEFT (LORA) finetune for Binary Question Answering Task
- Use OpenAI token probability prediction for eval / ranking

Instruction Input	
Task Instruction:	Given the user's historical interactions, please determine whether the user will enjoy the target new movie by answering "Yes" or "No".
Task Input:	User's liked items: GodFather. User's disliked items: Star Wars. Target new movie: Iron Man
Instruction Output	
Task Output:	No.

TALLRec: An Effective and Efficient Tuning Framework to Align Large Language Model with Recommendation - 2023

3. LLM for Scoring / Ranking as Binary Question Answering Task

Table 3: Performance comparison between conventional sequential recommendation baselines and TALLRec under different few-shot training settings. The reported result is the AUC multiplied by 100, with boldface indicating the highest score. ‡: significantly better than all baselines with t-test p<0.01.

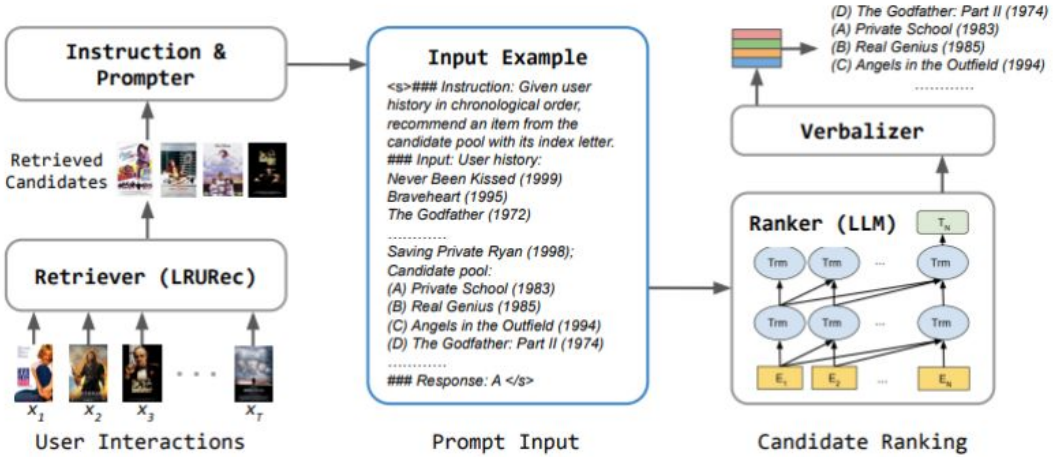
	Few-shot	GRU4Rec	Caser	SASRec	DROS	GRU-BERT	DROS-BERT	TALLRec
movie	16	49.07	49.68	50.43	50.76	50.85	50.21	**67.24‡**
	64	49.87	51.06	50.48	51.54	51.65	51.71	**67.48‡**
	256	52.89	54.20	52.25	54.07	53.44	53.94	**71.98‡**
book	16	48.95	49.84	49.48	49.28	50.07	50.07	**56.36**
	64	49.64	49.72	50.06	49.13	49.64	48.98	**60.39‡**
	256	49.86	49.57	50.20	49.13	49.79	50.20	**64.38‡**

Read more:

- PromptRec: Towards Personalized Cold-Start Recommendation with Prompts (2023)
- Collm: Integrating collaborative embeddings into large language models for recommendation (2023)
- Prompt4NR: Prompt Learning for News Recommendation (2023)
- ReLLa: Retrieval-enhanced Large Language Models for Lifelong Sequential Behavior Comprehension in Recommendation (ReLLa) (2024)

3. LLM for Item List Generation Task

- **Open-set item generation** : *hallucination problem*
- **Closed-sed item generation**



LlamaRec: Two-Stage Recommendation using Large Language Models for Ranking - 2023

3. LLM for Item List Generation Task

	ML-100k					Beauty					Games				
	NARM	BERT	SAS	LRU	LlamaRec	NARM	BERT	SAS	LRU	LlamaRec	NARM	BERT	SAS	LRU	LlamaRec
M@5	0.0298	0.0188	0.0333	0.0390	**0.0440**	0.0289	0.0246	0.0336	0.0376	**0.0385**	0.0479	0.0422	0.0515	0.0533	**0.0600**
N@5	0.0359	0.0232	0.0409	0.0468	**0.0543**	0.0342	0.0298	0.0397	0.0435	**0.0450**	0.0576	0.0512	0.0617	0.0640	**0.0714**
R@5	0.0544	0.0368	0.0641	0.0705	**0.0852**	0.0503	0.0457	0.0582	0.0614	**0.0648**	0.0874	0.0788	0.0930	0.0966	**0.1061**
M@10	0.0332	0.0244	0.0378	0.0491	**0.0529**	0.0321	0.0276	0.0371	0.0417	**0.0428**	0.0541	0.0478	0.0583	0.0598	**0.0671**
N@10	0.0441	0.0366	0.0522	0.0705	**0.0759**	0.0420	0.0372	0.0481	0.0533	**0.0554**	0.0729	0.0649	0.0783	0.0800	**0.0887**
R@10	0.0801	0.0785	0.0993	0.1426	**0.1524**	0.0746	0.0686	0.0844	0.0916	**0.0971**	0.1351	0.1214	0.1446	0.1463	**0.1599**

Read more:

- DRDT: Dynamic Reflection with Divergent Thinking for LLM-based Sequential Recommendation (2023)
- LiT5: Scaling Down, LiTting Up: Efficient Zero-Shot Listwise Reranking with Seq2seq Encoder-Decoder Models (2023)
- Agentcf: Collaborative learning with autonomous language agents for recommender systems (2023)
- RecPrompt: A Prompt Tuning Framework for News Recommendation Using Large Language Models (2023)

4. LLM for User Interaction

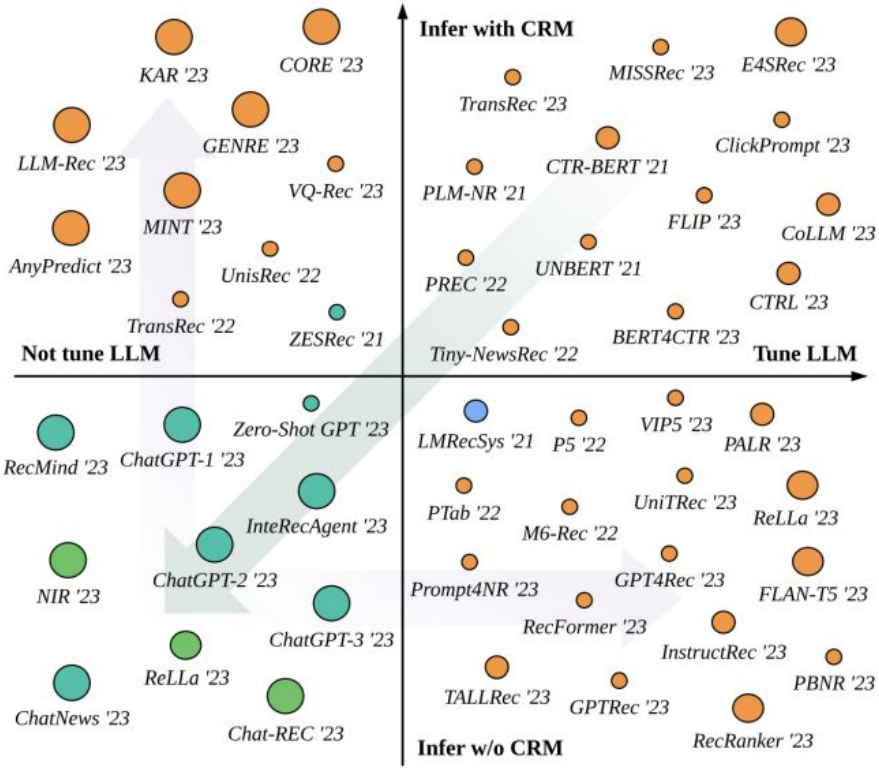
Recommending is a **one-turn interaction** process

LLM could engage in **real-time interactions** with the users to gather more nuanced natural language feedback about their preferences :

- Task-oriented User Interaction
- Open-ended User Interaction

=> Not really practical on production (read more in survey)

Evolution of LLM (tune or not) with RecSys (infer with / without)



Defeated Baseline

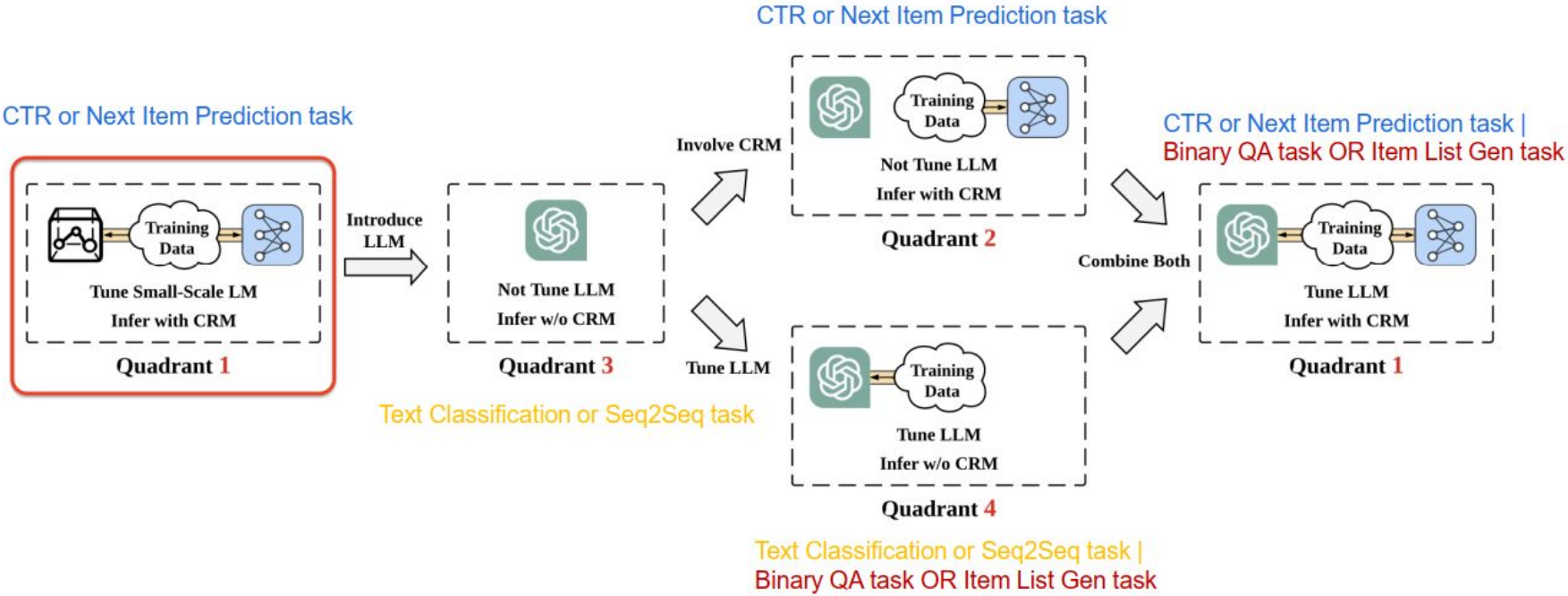
- Random
- Popularity
- MF
- MLP
- Attention-based

Size of LLM

- <1B
- 1B-10B
- 10B-100B
- >100B

Development Trajectory

Evolution of LLM (tune or not) with RecSys (infer with / without)



Summary: Challenges of using LLM for Real World Applications

- Efficiency
  - Training efficiency: LORA finetuned efficiently on Llama 7B, GPU RTX 3k
  - Inference Latency: PEFT only solve memory usage problem
    - **Pre-computing and caching**
    - **Reduce the size of the model:** Distillation, Pruning, Quantization
- Effectiveness
  - Long Text Modeling: **filter, select, and arrange the key textual information**
  - ID Indexing & Modeling: currently both works, best approach is still an open research problem
- Risks
  - Fairness: user bias (gender, age, race,…) & item bias
  - Hallucination
  - Privacy
  - Explainability

Resources :

- Exploring the Impact of Large Language Models on Recommender Systems: An Extensive Review, arxiv 2024, [paper].
- A Survey on Large Language Models for Recommendation, arxiv 2023, [paper].
- How Can Recommender Systems Benefit from Large Language Models: A Survey, arxiv 2023, [paper].
- Recommender Systems in the Era of Large Language Models (LLMs), arxiv 2023, [paper].